claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a8227855a44240c6c.jsonl`
- SHA-256: `3a918bcd1fd060db79c2935b4b458895d63c61269b1c13970f40896fafbaa7ae`
- Source modified: `2026-07-02T04:58:28+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T04:45:39.418Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.
REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).
AUDIT FINDINGS YOU ARE FIXING: R1-17 (scratch never reclaimed) + R1-03 overlap ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.
TASK BRIEF:
In C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer: reclaim remote-input scratch. (1) backend/storage/__init__.py:89 acquire_local_input mkdtemp-per-call (rally-input-*): give worker job paths a per-job scope that deletes the scratch dir in a finally (the worker owns the job lifecycle ? check backend/worker/__main__.py + backend/api/job_worker.py call sites); (2) backend/compute/cloud_run.py _read_json marker reads: use tempfile.TemporaryDirectory context; (3) add an age-based startup sweep for orphaned rally-input-*/rally-worker-* dirs older than N days (default 3) in the server boot path, default-ON but logging what it removes, with a config kill-switch. DO NOT touch the /api/source path (a sibling PR adds a keyed cache there ? avoid backend/main.py get_source entirely to prevent conflicts). Tests: finally-cleanup on success+failure, sweep age-threshold respects the kill-switch.

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):
1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master   (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.
COMMIT/PR CONVENTIONS:
- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.
QUALITY BAR:
- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md §0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.

- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:47.186Z)

70:### R1-03 [P1/M] critical-fix ? badminton-highlight-indexer
196:### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer

3. user (2026-07-02T04:45:48.260Z)

CLAUDE.md
README.md
__pycache__
artifacts
backend
backend.db
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
tests
tools
training

4. user (2026-07-02T04:45:56.298Z)

55   - evidence: gh pr view 390: state=OPEN, mergeable=CONFLICTING, mergeStateStatus=DIRTY, updatedAt=2026-06-26T14:37Z, 3 files (backend/main.py, backend/orchestration.py, mypy.ini). Since then >=8 PRs modified backend/main.py (#400, #405-#409, #412, #413 per git log --since=2026-06-25 -- backend/main.py). wc -l backend/main.py = 2198 vs the audit doc's '1448 lines' claim (docs/CODE_CLEANUP_AUDIT_2026-06-24.md:279); 18 @app. routes and 45 print() calls now live in it; backend.main still mypy-quarantined (mypy.ini:15). No backend/orchestration.py exists on master.
56   - why it matters: This is the last HIGH-severity item of the cleanup audit (P10, the only non-Phase-3 ? row in its §6 tracker) and it is compounding: every new endpoint PR lands in the god-module, widening the conflict surface and making the split more expensive each week. The 3-file extraction in #390 is now unmergeable.
57   - proposed fix: Close #390 and re-cut the orchestration.py extraction fresh from origin/master (small, ~1 session), or rebase it now. Then continue the split (CLI handlers -> backend/cli/, new /api/source|/api/assets|/api/capabilities routes -> backend/api/) before more endpoints accrete. Update the audit doc's stale '~1450 lines' sizing while at it.
58   - verifier notes: Minor sharpenings only: (a) git log --since=2026-06-25 --

backend/main.py actually shows 10 commits (adds #387 repo-wide ruff format ? itself a likely source of #390's conflicts ? and #380), so ">=8 PRs" is understated; (b) the mypy quarantine block is mypy.ini:15-16 ([mypy-backend.main] header at 15, ignore_errors at 16); (c) the audit doc's "1448 lines" statement is in the §Changelog 2026-06-25 "Accuracy fixes" bullet (~line 278), and the stale "~1450" sizing also appears at lines 69, 198, and 247 ? all four spots need the size refresh the fix proposes.
59   - verifier reasoning: Every load-bearing claim reproduced. (1) `gh pr view 390` returns state=OPEN, mergeable=CONFLICTING, mergeStateStatus=DIRTY, updatedAt=2026-06-26T14:37:18Z, exactly 3 files: backend/main.py (+19/-595), backend/orchestration.py (ADDED, +635), mypy.ini (+6). (2) Current working tree (branch master, HEAD e410136 = PR #413): `wc -l backend/main.py` = 2198; 1448?2198 is +51.8% ? the claimed 52%; the 1448 baseline is documented in docs/CODE_CLEANUP_AUDIT_2026-06-24.md changelog ("main.py size '~1300' ? '~1450' (now 1448 lines)", 2026-06-25 entry, ~line 278-279). (3) Not fixed: backend/orchestration.py does not exist in the tree, and git log --since=2026-06-25 -- backend/main.py shows the file only grew ? commits for PRs #413, #412, #409, #408, #407, #406, #405, #400 (the claimed 8; the log actually lists 10 incl. #387 ruff-format and #380). (4) grep counts on backend/main.py: 18 lines matching `^@app\.` and 45 `print(` occurrences ? exactly as claimed. (5) mypy.ini:15-16 = `[mypy-backend.main]` / `ignore_errors = True` (quarantine intact). (6) Audit doc §6 tracker line 198: P10 row "D1: Split main.py god-module (~1450 lines ? smaller modules)" is ?; the only other ? row is line 205 (P16+ Phase 3 bucket), so P10 is indeed the sole non-Phase-3 open item; D1 is rated HIGH at doc line 69. Severity P1 is honest for a tech-debt audit: the concrete failure is already materialized (the 635-line extraction in #390 is unmergeable today), and it compounds ? 8 endpoint-adding PRs landed in the god-module in the week since the PR was cut, all inside a mypy ignore_errors module, so each week of delay increases rework cost and unchecked-type surface. Not owner-gated: internal refactor, no serving go-live / corpus-promotion / spend / licensing implications ("P10" here is the audit tracker row, not Studio P10). Proposed fix (close-and-recut or rebase from origin/master, then continue the split per the doc's own plan at line 247, and fix the stale ~1450 sizing) is sane and correctly scoped; effort M is right given the fix includes continuing the split beyond the re-cut.
60
61   ### R1-02 [P1/M] critical-fix ? badminton-highlight-indexer
62   **/api/process re-runs the paid Gemini segmenter for an already-processed video (re-bill on refresh/'Try again') ? still unfixed at HEAD**
63   - anchor: `backend/main.py:1172` · tracked in: Khelsutra/badminton-highlight-indexer#340 (also #330) · finder: engine:correctness
64   - evidence: backend/main.py:1171-1172 computes video_id and `was_reingest = db.get_video(video_id) is not None`, but was_reingest is only used for the report/destroy-after-confirm; there is NO reuse/skip branch ? the segmenter always runs (main.py:1251-1255). Verified against gh: issue #340 OPEN ('re-runs the full (paid) segmenter anyway'), plus #330 (MD5 segmentation reuse) OPEN.
65   - why it matters: A page refresh or SPA 'Try again' on an already-indexed video (same MD5) re-bills Gemini (~$0.10/8-min match) and burns a multi-minute compute run for zero new information. Live-triggered once in the alpha (2026-06-22, per #340).
66   - proposed fix: In _execute_processing, when db.get_video(video_id) exists with status='processed' and existing play_segments (and the request doesn't explicitly force reprocess), return the stored segments as the job result instead of dispatching the segmenter; add a `force` flag to ProcessRequest for deliberate re-ingest.
67   - verifier notes: Severity P1 is honest: concrete failure scenario is a user-reachable one-click ("Try again" after the SPA's 10-min poll ceiling, live-triggered 2026-06-22 per #340) that silently re-bills a metered Gemini key ? and per #340 the M8 cost ledger does not meter Gemini yet, so the re-bill is invisible. Proposed fix is sane and correctly scoped: the skip condition must require status=="processed" AND existing play_segments (a prior "failed" record ? main.py:1196-1201 ? must still re-run), and the added `force` flag preserves the existing destroy-after-confirm re-ingest path (main.py:1233-1235, 1278). Not owner-gated: the fix reduces real cloud spend rather than incurring it, and the owner already specified the desired behavior + acceptance criteria in OPEN issue #340. Effort M is right (endpoint logic + ProcessRequest field + a no-Gemini-call regression test). One duplicate-tracking note: #330 covers the same reuse mechanism as an optimization; a fix should close both.
68   - verifier reasoning: Verified against origin/master at HEAD e410136 (pulled, "Already up to date"). (1) Code behaves as claimed: backend/main.py:1171-1172 computes `video_id = compute_video_id(local_video)` then `was_reingest = db.get_video(video_id) is not None`; the ONLY other use of `was_reingest` in `_execute_processing` is main.py:1327, where it is passed to `emit_indexer_report` as a report field. There is no reuse/skip/short-circuit branch: after validation passes (main.py:1210-1214 upserts status "processed"), the flow unconditionally dispatches segmentation ? cloud via `_maybe_dispatch_remote` (main.py:1241-1248) or in-process

`segmenter.process_video` (main.py:1251-1255). ProcessRequest (main.py:313-324) has no `force`/`reprocess` field (only video_path, player_pool, max_frames, segmenter, locale, compute). (2) Not fixed: `git log --oneline -15 -- backend/main.py` shows only unrelated commits (#413, #412, #409, #408, #407, #406, #405, #400, ...); no idempotency/reuse change. (3) Cost claim is real in served config: config.json:26 sets `"default_segmenter": "gemini"` (the paid path is the production default; the "motion" fallback at main.py:1181-1183 is only the code-level default). (4) Tracking verified via gh: issue #340 OPEN (title and body match the finding verbatim, including the 2026-06-22 live trigger and ~$0.10/8-min cost) and #330 OPEN (MD5 segmentation reuse).
69
70   ### R1-03 [P1/M] critical-fix ? badminton-highlight-indexer
71   **/api/source re-downloads the ENTIRE remote video from the bucket on every HTTP request (each browser Range request) into a fresh, never-deleted temp dir**
72   - anchor: `backend/main.py:646` · tracked in: ? · finder: engine:correctness
73   - evidence: backend/main.py:644-651: get_source calls storage.acquire_local_input(rec['filepath'], ...) on EVERY request; storage/__init__.py:89-90: for a non-local ref this does `tempfile.mkdtemp(prefix='rally-input-')` + full fetch ? a new dir + full download per call, never removed. Proxy warming is explicitly local-only (main.py:655-657), so a remote-source video never gets a proxy. FileResponse under fastapi>=0.111 (pyproject.toml:28) serves Range ? a browser scrubbing a <video> issues many separate requests to the same URL.
74   - why it matters: Direct cloud-egress cost leak plus a disk-fill denial-of-service of the appliance, reachable through a supported flow (remote-registered assets + P8 annotate-in-UI).
75   - proposed fix: Cache the fetched local copy keyed by video_id (e.g. under <output>/proxies or a per-video workspace source/ dir) and reuse it across requests; extend proxy warming to remote sources; delete or reuse the scratch dir instead of mkdtemp-per-request.
76   - verifier notes: Path correction: the storage citation is backend/storage/__init__.py:89-90 (not storage/__init__.py). Sharper failure scenario: store a 4 GB match via POST /api/assets with an s3://gs:// medium (P4, #409), open the P8 annotate UI, scrub ~10 times ? ~40 GB bucket egress (~$4-5 real cost) and ~40 GB of never-deleted rally-input-* temp dirs per session; each seek also stalls behind a full multi-GB download, so the feature is effectively unusable for remote sources. Additional color for the fix: the engine free-space guard (require_free_bytes, #407) is not applied to this download path, and the cached copy should live under a guarded root; the finding's proposed fix (cache keyed by video_id, reuse across requests, extend proxy warming to remote) is sane and correctly scoped.
77   - verifier reasoning: backend/main.py:646 calls storage.acquire_local_input(rec["filepath"], ...) on every GET /api/source/{video_id} when no proxy exists (proxy check main.py:641-642). backend/storage/__init__.py:89-90: non-local refs get a fresh tempfile.mkdtemp(prefix="rally-input-") + full fetch per call; docstring (lines 80-82) admits the copy "relies on OS temp reclamation" (nonexistent on Windows %TEMP%), and a repo-wide grep for "rally-input" shows no cleanup path. Remote filepaths are a supported state: POST /api/assets registers the cloud stored_uri as filepath (main.py:838, 855; comment at main.py:1680 lists gs://, s3://, gdrive://). Proxy warming is explicitly local-only (main.py:655-657, docstring 628-629 "remote-source proxying is a follow-up"), so remote sources never escape the full-fetch path. Installed fastapi 0.136.1 / starlette 1.0.0 FileResponse honors Range, so a scrubbing <video> issues many independent requests, each a full-object download. git log -15 -- backend/main.py (through #413) shows no fix after #406 introduced this. Not owner-gated: the fix is plain engineering that reduces cloud spend.
78
79   ### R1-04 [P1/M] tech-debt ? badminton-highlight-indexer
80   **Import cycles: extracted API modules reach back into backend.main globals at call time; config layer imports pipeline layer**
81   - anchor: `backend/api/job_worker.py:75` · tracked in: docs/CODE_CLEANUP_AUDIT_2026-06-24.md P5 note + D1/P10 follow-up (line 221) · finder: engine:structure
82   - evidence: AST scan of backend/ found 5 strongly-connected components. (1) backend.main <-> backend.api.job_worker <-> backend.api.uploads: job_worker.py:75,98,214,234,251 and uploads.py:47 do call-time 'import backend.main as _main' to read main.py's module globals db_instance/global_config (main.py:279-280); main.py:292 re-exports job_worker names back onto itself. Documented as a transitional shim (CODE_CLEANUP_AUDIT P5 note, line 221) but it makes main.py load-bearing for every extracted module and keeps backend.main AND backend.worker.__main__ mypy-quarantined (mypy.ini:15-23). (2) Layering inversion: backend/config/models.py:249 imports segmenter_registry from backend.pipeline.segmenters.base inside a validator, while base.py:15 (_normalize_config) imports AppConfig back from

config.models ? config validation now depends on pipeline registration side effects. (3-5) eval-package cycles (golden_fixtures<->golden_real_fixtures, ablation<->ablation_pdf, annotations<->gt_loader) are lazy and explicitly documented (golden_fixtures.py:1005-1028) ? lower priority.
83   - why it matters:
84   - proposed fix: Kill the shim as the concrete step-2 of D1: move db_instance/global_config onto app.state (create_app already exists per P5) and pass config/db explicitly to job_worker/uploads, then lift backend.main and backend.worker.__main__ out of mypy quarantine. For the layering inversion, move the segmenter-name validation out of the pydantic validator into a startup check (backend/config/startup.py) so config/ no longer imports pipeline/.
85   - verifier notes: Mechanism correction to the evidence: job_worker's five call-time imports do NOT read db_instance/global_config ? job_worker already takes config/db as explicit params (job_worker.py:89,205,230,247; mypy.ini:14 notes it was lifted from quarantine) and resolves FUNCTION seams (ProcessRequest, _execute_compile, _execute_processing, JobWaiting, _arm_wake_timer, _get_executor, _MAX_DRAIN_WAKE_SEC) through the backend.main module object per the documented test-monkeypatch contract (job_worker.py:8-16, main.py:286-291). Only uploads.py:49 reads a main global (global_config). So the app.state migration for db/config is mostly about uploads + main.py itself; the harder remaining work is redesigning the monkeypatch seam contract and its tests ? effort is top-end M. Sharper failure scenario: any alternative entrypoint (new console_script, uvicorn factory) that builds the app without running main() leaves backend.main.global_config=None and uploads silently falls back to default upload dir/size limits (uploads.py:49-52), ignoring operator security config ? same class as the already-documented python -m backend.main incident (main.py:2188-2195) that hung every queued job. Layering inversion is slightly worse than claimed: the validator's lazy import (models.py:249) transitively executes segmenters/__init__.py:1-7, importing ALL concrete segmenters, so constructing AppConfig anywhere drags the full pipeline in; and SegmenterRegistry.register (base.py:66-70) re-enters AppConfig construction during registration with errors swallowed by base.py:19's bare except.
86   - verifier reasoning: Verified in current tree (HEAD e410136, pulled clean). Cycle 1: backend/main.py:292-302 imports backend.api.job_worker at module load (re-exports); job_worker.py:75,98,214,234,251 do call-time 'import backend.main as _main' (grep-confirmed all 5); uploads.py:47-49 call-time imports and reads _main.global_config; globals defined main.py:279-280, set at main.py:101-103 and 1981-1982; mypy.ini:15-16 quarantines backend.main, :22-23 backend.worker.__main__. Cycle 2: config/models.py:244-256 validator imports segmenter_registry from pipeline/segmenters/base; base.py:14-18 imports AppConfig back in _normalize_config; importing base initializes segmenters/__init__.py:1-7 which imports every concrete segmenter (motion.py:243 etc. register as import side effects), so AppConfig construction depends on pipeline import health ? and register() (base.py:66-70) re-enters AppConfig via segmenter_cls(None, {}) with failures swallowed by base.py:19 bare except. Eval cycles lazy+documented (golden_fixtures.py:1008-1013). Not already fixed: git log for the four files shows only feature commits (#405-#413). Severity honest: the failure class already bit ? main.py:2188-2195 documents the python -m backend.main dual-module incident (backend.main.db_instance=None, drain crashed silently, jobs hung 'queued' forever), patched only for that entry path (main.py:2196-2198); uploads.py:49-52 getattr-defaults mean any entrypoint that skips main() silently runs uploads with default dir/limits. Tracked_in accurate: audit doc line 221 is verbatim the P5 shim note. Not owner-gated: only owner question is D1 phase sequencing, none of the listed gates. Fix sane: create_app exists (main.py:2184), handlers already use app.state (mypy.ini:12-13), backend/config/startup.py exists as the startup-check home.
87
88   ### R1-05 [P1/M] tech-debt ? badminton-highlight-indexer
89   **Path-jail rejects gs://,s3://,gdrive-api:// before scheme resolution ? bucket ingest impossible on the exposed alpha box (issue #327 still open)**
90   - anchor: `backend/main.py:1376` · tracked in: https://github.com/Khelsutra/badminton-highlight-indexer/issues/327 · finder: engine:security-config
91   - evidence: main.py:1376 calls `validate_input_path(req.video_path, allowed_root=allowed_root)` BEFORE `storage.resolve_input_ref` at main.py:1379; backend/utils/paths.py:45-55 `resolve_under_root` realpaths the raw URI so `gs://bucket/x.mp4` escapes the jail -> 400. Startup check startup.py:106-119 makes the jail mandatory when exposed (EXPOSED_NO_VIDEO_ROOT), so an exposed engine can never ingest from a bucket. gh: issue #327 OPEN (verified 2026-07-02), proposing scheme-aware validation.
92   - why it matters: Blocks the alpha CUJ (large-video ingest via gs:// instead of 500 MB browser uploads) and is a stale-open thread since before 06-26; the fix must be done carefully to avoid reopening the arbitrary-local-file hole the jail exists for.

93   - proposed fix: Implement #327 option (b): jail only after resolve_input_ref, applying resolve_under_root exclusively to backend=='local' refs; keep traversal/symlink/absolute rejection for local paths and add both allow (bucket URI accepted) and reject (local traversal still 400) tests, updating tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri.
94   - verifier notes: Minor corrections: (1) startup check path is backend/config/startup.py:106-119, not startup.py (lines exact). (2) The "500 MB browser uploads" figure in why_it_matters comes from issue #327's body; the repo's actual browser-upload cap is security.max_upload_mb default 4096 MB (backend/api/uploads.py:51, backend/config/models.py:339) ? 500 MB is the Gemini provider per-clip cap (backend/providers/gemini.py:23). Immaterial to the finding (multi-hour matches exceed 4 GB anyway; docs/VIDEO_LOCALITY_MODEL.md:26 "The 10 GB problem is real"). (3) Fix scope additions: there is a second resolve call-site (CLI --video-path, see tests/test_api_endpoints.py:922) ? verify whether it needs the same scheme-aware treatment; ensure unknown schemes still ValueError?400 via resolve_input_ref as the sole scheme gate. (4) Caution, not a gate: the change relaxes a deliberately pinned security contract on the exposed box; #327 records the owner's explicit want, but the option (a) vs (b) choice and the PR should get owner sign-off before landing since it alters exposed-serving posture.
95   - verifier reasoning: Personally reproduced from current code. (1) Ordering: backend/main.py:1376 calls validate_input_path(req.video_path, allowed_root=allowed_root) BEFORE storage.resolve_input_ref at main.py:1379-1381; ValueError maps to HTTP 400 at main.py:1382-1383. (2) Mechanism: backend/utils/paths.py:58-68 routes any request with allowed_root set through resolve_under_root, which os.path.realpath()s the RAW string (paths.py:47) ? 'gs://bucket/x.mp4' realpaths to a cwd-relative path, commonpath != root (paths.py:50-54) ? ValueError ? 400. The code itself documents this at main.py:1387-1389: "a configured allowed_video_root (hosted mode) rejects a non-local URI as out-of-root". (3) Pinned by test: tests/test_api_endpoints.py:903-912 asserts gs://, s3://, gdrive:// all ? 400 when allowed_video_root is set ("Pins the security contract"). (4) Exposure mandate: backend/config/startup.py:106-119 emits LEVEL_ERROR EXPOSED_NO_VIDEO_ROOT when edge_auth_enabled and allowed_video_root unset ? so an exposed engine must have the jail, making bucket ingest impossible there. (5) Not fixed: git log -15 -- backend/main.py (head e410136, #413) shows no commit addressing this; behavior present in current tree. (6) gh issue view 327: state OPEN, created 2026-06-22, title/body match the finding exactly, including the owner's explicit want ("ingest large videos via gs:// on the hosted box") and options (a)/(b). Concrete failure: exposed alpha box (edge_auth_enabled=true, allowed_video_root=/jail per boot requirement) receives POST /api/process {"video_path":"gs://bucket/match.mp4"} ? 400 "path ... escapes the allowed root"; only ingest path left is the capped browser upload (backend/api/uploads.py:51, security.max_upload_mb) ? blocks the tracked alpha large-video CUJ. Severity P1 honest (owner-wanted CUJ blocked, open since 2026-06-22, workaround capped/painful per docs/VIDEO_LOCALITY_MODEL.md:19-26). Proposed fix (option b: jail only backend=='local' after resolve_input_ref, keep null-byte check universal and traversal/symlink jail for local, add allow+reject tests, update the pinned test) matches the issue's own proposal and is correctly scoped; effort M is right.
96
97   ### R1-06 [P1/S] doc-rot ? khelsutra (avidullu/khelsutra)
98   **Docs claim the retired droplet alpha is LIVE (droplet deleted 2026-06-25)**
99   - anchor: `NEXT_STEPS.md:12` · tracked in: partially in issue #97 (item 2: update DEPLOY_ALPHA.md); the NEXT_STEPS.md droplet claim is untracked · finder: khelsutra:issues-triage
100   - evidence: NEXT_STEPS.md:12-13 ('Where we are (updated 2026-07-01)') still says the site is 'live + browsable on the DO droplet -> http://168.144.126.176:8787'; deploy/DEPLOY_ALPHA.md:3 says 'Status: LIVE (deploy live-verified on the droplet 2026-06-22)' and :29 'This runbook ships the split topology'; docs/LOCAL_APPLIANCE_ARCHITECTURE.md:176 'The alpha stand-up is now LIVE (2026-06-22)'. Project memory (MEMORY.md:63, session-handoff.md:19): droplet claude-do-rally-test/168.144.126.176 RETIRED+deleted 2026-06-25; alpha paused pending Cloud Run. nslookup shows api./app.khelsutra.guru still resolve via Cloudflare to a tunnel with no origin (pending CNAME/Access cleanup).
101   - why it matters:
102   - proposed fix: One doc PR: mark DEPLOY_ALPHA.md status RETIRED/PAUSED with a pointer to the Cloud Run plan, fix NEXT_STEPS.md 'Where we are' (droplet gone, site is CF-Workers-only), and correct LOCAL_APPLIANCE_ARCHITECTURE.md:176. Also note the single-origin repoint that superseded the split topology (issue #97). The 2026-07-01 refresh (#139) touched docs but kept the stale claim, so this will keep misleading ramp-up reads.
103   - verifier notes: Minor anchor correction: the 'Where we are (updated 2026-07-01)' header is NEXT_STEPS.md:5; the droplet-live claim spans lines 12-13 (the finding's line 12 anchor is correct for the URL text). Severity P1 stands: NEXT_STEPS.md's 2026-07-01 stamp is

NEWER than the handoff's 2026-06-25 correction, so ramp-up readers reconciling the two will trust the wrong doc; worse, DEPLOY_ALPHA.md §1 (line 35) directs deploying the engine + secrets onto 168.144.126.176 ? an IP the owner no longer controls and DO can recycle. Fix is sane, doc-only, and not owner-gated (it records already-made owner decisions: retirement 2026-06-25 + single-origin repoint per issue #97); one caution ? the fix should point to the Cloud Run move as 'paused pending' rather than declare a new serving decision, which would be owner-gated. Did not independently re-verify the finder's nslookup/DNS claim; the finding stands without it.
104   - verifier reasoning: Working tree at HEAD f9eea14: NEXT_STEPS.md:5 'Where we are (updated 2026-07-01)' + :12-13 'live + browsable on the DO droplet -> http://168.144.126.176:8787 (systemd khelsutra-site)'; deploy/DEPLOY_ALPHA.md:3 'Status: LIVE (deploy live-verified on the droplet 2026-06-22...)' + :29 'This runbook ships the split topology'; docs/LOCAL_APPLIANCE_ARCHITECTURE.md:176 'The alpha stand-up is now LIVE (2026-06-22): the engine booted on the droplet (168.144.126.176)'. Ground truth: memory/khelsutra-site-droplet-parity.md:3,10-14 (droplet DELETED 2026-06-25, data archived, doctl delete done, monitor removed via PR #117), session-handoff.md:19 ('the prior LIVE alpha on the droplet is GONE ? alpha paused pending the Cloud-Run move'), MEMORY.md:63. Not fixed: git log shows the last commit touching all three files is 9223ba9 (#139, the 2026-07-01 refresh) and the stale text persists at HEAD. Tracking verified: gh issue 97 is OPEN, item 2 covers only DEPLOY_ALPHA.md (single-origin update); the NEXT_STEPS.md droplet claim is untracked. Issue #97 also confirms split topology was superseded by the 2026-06-22 single-origin repoint, so DEPLOY_ALPHA.md:29 is doubly stale.
105
106   ### R1-07 [P1/M] critical-fix ? khelsutra (avidullu/khelsutra)
107   **Issue #111 still fully valid: refresh loses the in-flight job; progress static for minutes**
108   - anchor: `web/src/hooks/useIngestJob.ts:117` · tracked in: khelsutra#111 · finder: khelsutra:issues-triage
109   - evidence: useIngestJob.ts:117-128 ? job_id lives only in the poll closure (no localStorage/URL persistence); App.tsx:71-73 seeds state only from ?video= (set on done, App.tsx:153). A refresh mid-job lands the user back on the ingest form while the server job keeps running. Progress is the coarse free-text job.progress (useIngestJob.ts:106); no heartbeat/animation beyond a spinner chip (IngestPanel.tsx:337-358).
110   - why it matters:
111   - proposed fix: SPA half is Claude-doable now: persist the active job_id (?job=<id> + localStorage) and on load resume polling to the existing ?video= handoff; add elapsed-time/heartbeat copy. Engine half (granular 'segmenting chunk 3/7' from backend/pipeline/segmenters/gemini.py via the job-progress hook) is a small cross-repo change. Per the issue this is 'the most likely way an early tester loses a working job'.
112   - verifier notes: Failure scenario, sharpened from issue #111's field report: an ~8-min upload spends ~5 min in Gemini segmentation with near-static "processing" text; the tester refreshes (natural when nothing moves), lands on the ingest form with no trace of the running job, and either re-submits ? a duplicate job, potentially on paid cloud GPU via the compute picker ? or abandons believing it failed; the finished video only appears later in the library with no pointer back. Fix note: the backend is already refresh-safe (jobs persisted in SQLite), so the SPA half (persist ?job=/localStorage, resume polling into the existing ?video= handoff, heartbeat/elapsed-time copy) is standalone and Claude-doable now; only the granular-progress half touches the sibling badminton-highlight-indexer repo.
113   - verifier reasoning: useIngestJob.ts:117-128 ? job_id is a closure-local (`let jobId` at :117, used only by poll(:128)); grep of web/src confirms no localStorage/URL persistence of any job id (only annotate flag, wizard key, i18n lang). App.tsx:71-73 seeds videoId solely from ?video=, written only on job-done via loadVideo (App.tsx:150-154 ? useIngestJob.ts:96-98), so a mid-job refresh renders the ingest form (App.tsx:338-343) while the server job keeps running. Progress is coarse free-text job.progress (useIngestJob.ts:106) shown as a spinner chip (IngestPanel.tsx:337-348), no heartbeat/elapsed time. git log -15 on both files (latest #140, #138, #128?) shows no fix; issue khelsutra#111 is OPEN and documents the identical gaps from a live 2026-06-22 alpha run. Engine path backend/pipeline/segmenters/gemini.py exists in the sibling badminton-highlight-indexer repo as the finder scoped it. Not owner-gated: pure client resiliency, no go-live/corpus/licensing/spend/legal decision.
114

5. user (2026-07-02T04:45:57.032Z)

190   - evidence: §7 tracker (lines 63-73): P1-P8 all ?. Verified in code: grep 'build_stitch_plan' across backend/ and tests/ returns nothing (P2's pure stitch-plan seam was never factored out of backend/pipeline/stitcher/compiler.py), and tests/test_golden_fixtures.py

has no stitch-plan snapshot. Minor rot: the P0 row (line 65) still shows ? '(this PR)' though the doc merged as #383 (commit 1283e18, 2026-06-25) and the header still says 'DRAFT'.
191   - why it matters: This gate exists precisely to bless genuine (non-AST-identical) refactors as quality-neutral ? i.e., exactly the class of change as the stalled #390 main.py split. Executing P1-P4 first would de-risk redoing the god-module split; leaving it a week-stale locked-decisions doc means refactors continue with no behavioral proof.
192   - proposed fix: Start §7 P1+P2 (canonical decision-snapshot schema + factor pure build_stitch_plan() out of compiler.py) as the next Claude-doable engine work; flip the P0 row to ? #383 and status to IN PROGRESS in the same PR. Ideally land P1-P4 before re-cutting the main.py split.
193   - verifier notes: One factual correction to the evidence: P8 (line 73) is ? (blocked/owner-gated), not ? ? "P1-P8 all ?" should read "P1-P7 all ?, P8 ?". Substance unaffected: zero of nine tracker rows show any progress. Sharper failure scenario: #390 gets re-cut and merged as a "quality-neutral" god-module split with no behavioral proof ? the only offline gate today is the segmentation-only characterization snapshot (test_replay_matches_committed_expected), so a subtle behavior change in stitch/concat-plan construction (clip order, cut points, padding) ships undetected until a GPU nightly or a user-visible reel regression. Effort L is honest for the proposed P1-P4 scope; the minimal P0-row/status flip alone is S and could be split out.
194   - verifier reasoning: Personally verified in the current working tree (master, clean). (1) docs/DECISION_SNAPSHOT_REGRESSION.md:8 still reads "Status: `DRAFT` ... LOCKED (2026-06-25) ... Last updated: 2026-06-25"; line 65 P0 row is "? | (this PR)" even though the doc merged as PR #383 (gh: MERGED 2026-06-25T17:47:44Z; sole commit 1283e18 touching the file ? `git log --oneline -15 -- docs/DECISION_SNAPSHOT_REGRESSION.md` returns exactly one commit, so nothing fixed it since). (2) Zero code shipped: grep for build_stitch_plan across the repo hits only the doc itself and docs/README.md; backend/pipeline/stitcher/compiler.py:24 contains only `class VideoCompiler` with no pure plan-builder seam; grep -i stitch under tests/ returns 11 files NOT including tests/test_golden_fixtures.py, confirming no stitch-plan snapshot exists; repo-wide grep for stitch_plan|decision_snapshot matches only docs. Tracker rows at lines 66-72 (P1-P7) are all ? with PR "?". (3) Why-it-matters context holds: gh shows PR #390 ("refactor(D1/P10): extract orchestration.py from main.py (god-module split, step 1)") is still OPEN ? exactly the non-AST-identical refactor class this gate (D11, line 40) is designed to bless. (4) Not owner-gated: the only owner-gated rows are P0 (already satisfied ? line 82 "[x] D1?D11 locked by the owner (2026-06-25)") and P8 (line 73); P1-P7 are marked "Gated? No", and the proposed fix touches only non-gated rows and none of the listed gate categories.
195
196   ### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer
197   **Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud_run marker reads) ? 'OS temp reclamation' premise is false on Windows**
198   - anchor: `backend/storage/__init__.py:89` · tracked in: ? · finder: engine:correctness
199   - evidence: storage/__init__.py:80-90: acquire_local_input docstring says the scratch copy 'is left under the system temp dir ... and relies on OS temp reclamation'. Call sites that leak one full-video copy per run: _execute_processing (main.py:1168-1171), _execute_compile (main.py:1683-1686), create_asset (main.py:791-799), get_source (main.py:646). cloud_run.py:219-222 additionally leaks a cr-read-* mkdtemp per marker/report read. No janitor exists (grep for rally-input-/cleanup found none); engine issue #301 covers only the upload_dir retention sweep, not temp scratch.
200   - why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.
201   - proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output_base(cfg)/tmp with an age-based sweep at startup; make cloud_run._read_json use TemporaryDirectory.
202   - verifier notes: Two sharpenings. (a) get_source (main.py:646) cannot use a simple finally-delete: the FileResponse streams from the scratch file after the handler returns, so deletion there needs the age-based-sweep option (or a keyed scratch cache reused across views) ? the finder's second proposed option covers it; per-job finally scopes fit _execute_processing/_execute_compile/create_asset. (b) Add a fifth leaking call site to the fix scope: the CLI single-shot path at main.py:2056. Sharper failure scenario: with input_backend=gdrive-api/gcs, each processed match leaks one full multi-GB copy into %TEMP% per process job, another per compile, one per /api/assets source_uri registration (fetched solely to hash), and one per /api/videos/{id}/source view ? until require_free_bytes(tempfile.gettempdir()) at main.py:789/1402 starts rejecting all further remote ingests at stage="fetch" even though the disk is full of the tool's own dead scratch.
203   - verifier reasoning: Personally verified in the current tree. (1)

backend/storage/__init__.py:80-82 docstring says the scratch copy "is left under the system temp dir ? and relies on OS temp reclamation"; line 89 does tempfile.mkdtemp(prefix="rally-input-") and line 90 fetches into it ? no deletion. (2) Every claimed call site leaks: _execute_processing main.py:1168-1171 (no rmtree anywhere in main.py ? grep confirmed rmtree exists only in eval/golden_real_fixtures.py, pipeline/stitcher/compiler.py, pipeline/detectors/*); _execute_compile main.py:1683-1686; create_asset main.py:791-799 fetches a full video copy just to MD5-hash it (main.py:805) then returns at 865-875 without deleting local_working; get_source main.py:646 fetches per-request and serves it via FileResponse (line 660) ? a per-view full-video leak for remote sources. Plus a fifth site the finder missed: CLI single-shot main.py:2056. cloud_run.py:219-222 _read_json mkdtemps "cr-read-" per marker/report read, no cleanup. (3) No janitor: tools/cleanup_caches.py scans only WSL ~/clips, the output/ tree, and security.upload_dir (docstring lines 3-5, classify() lines 52-97) ? never tempfile.gettempdir(); its "tmp*" rule (line 79-80) applies only inside those trees. (4) Not fixed: git log for storage/__init__.py tops out at b6214e5 (repo-wide ruff format); gh issue 301 confirmed to cover only the T11 upload-retention scheduled job, not temp scratch. (5) Guard interplay real: require_free_bytes(tempfile.gettempdir(), ?, stage="fetch") at main.py:789 and main.py:1402 ? leaked copies shrink exactly the headroom the fetch guard checks, so accumulation converts into false fetch rejections. Windows does not auto-reclaim %TEMP% by default, so the docstring premise is false on the shipping platform. Severity P2 honest: affects only remote-input flows (local paths are identity passthrough, __init__.py:85-88), slow-burn, no data loss. Not owner-gated ? pure disk hygiene, no strategy/spend/ToS/corpus decision.
204
205   ### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206   **GDriveApiStorage.fetch_to_local buffers the entire file in RAM (get_media().execute()); put() deletes the old object before the new upload succeeds**
207   - anchor: `backend/storage/gdrive_api.py:144` · tracked in: ? · finder: engine:correctness
208   - evidence: gdrive_api.py:143-148: `data = self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole object is materialized as one bytes value (no MediaIoBaseDownload streaming). gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)` BEFORE `files().create(...)` ? delete-then-create is not atomic.
209   - why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it goes live for the reel round-trip.
210   - proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks; for put(), create the new file first (or upload to a temp name) and delete the old one only after success.
211   - verifier notes: File/line accurate (line 144 is the get_media().execute() call). Failure scenarios: (1) fetch_to_local on a multi-GB source/reel buffers the full file in RAM on a memory-limited Cloud Run box ? OOM kill; (2) re-put() of a reel to an existing path deletes the old file, then the resumable create fails (network/quota) ? the reel is gone with no copy at either end. Fix refinement: for put(), prefer `files().update(fileId=existing, media_body=...)` when a file already exists ? atomic in-place content replacement that preserves the file id and avoids both the delete-first data-loss window and the duplicate-(name,parent) window that create-first-then-delete would open (the file's own comment at lines 160-162 notes _resolve_id returns the first match among duplicates). Download fix as proposed: MediaIoBaseDownload chunked streaming.
212   - verifier reasoning: backend/storage/gdrive_api.py:143-148 ? fetch_to_local does `data = self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)`: the whole object is returned as a single bytes value; no MediaIoBaseDownload/chunked loop exists in the file (only MediaFileUpload for uploads, lines 78-84). Docstring (lines 3-6, 19) states this backend serves videos/reels on headless Cloud Run/GPU hosts, so multi-GB buffering is a genuine OOM risk. backend/storage/gdrive_api.py:163-170 ? put() resolves `existing`, calls `files().delete(fileId=existing).execute()`, and only then `files().create(...)`: a create failure after the delete loses the prior object (non-atomic overwrite). git log -- backend/storage/gdrive_api.py shows only formatting/isort commits (b6214e5, f523d3a) since the introducing commit 819e395 (M12, PR #338) ? not fixed. Not owner-gated: the module's owner gate (lines 14-17) covers the per-user OAuth flow, not this code's download/upload mechanics.
213
214   ### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer
215   **config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it**
216   - anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness
217   - evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global_config.output.output_dir = resolve_output_dir(args.output_dir)` ?

the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir overrides this at startup' ? implying only when given). db_path derives from it (main.py:1995-1997).
218   - why it matters: A documented config knob that is a no-op on the main entry path, with data-location consequences (perceived data loss / split state).
219   - proposed fix: Change argparse default to None and only override when args.output_dir is not None (falling back to the config value through resolve_output_dir).
220   - verifier notes: Path correction: the documented knob is at backend/config/models.py:274-276, not backend/models.py (which does not exist). Sharper failure scenario: user sets output.output_dir: "D:/rally-data" in config.json and restarts `indexer --app`; main.py:1992 resets the dir to resolve_output_dir("output"), a fresh empty DB is created there (main.py:1995-1997) so the UI shows zero videos (perceived data loss), while run_eval.py/run_regression.py via backend/config/__init__.py:41-42 resolve the DB at D:/rally-data ? server and eval tooling disagree on DB location. Fix scope addition: the trim path at backend/main.py:1927-1929 also calls resolve_output_dir(args.output_dir) before config load and must handle a None default (fall back to the loaded config value there too, or to "output" since global_config isn't initialized yet at that point). Effort stays S.
221   - verifier reasoning: Read current tree. backend/main.py:1837-1841 defines --output-dir with default="output". backend/main.py:1982 loads the typed config; backend/main.py:1992 then unconditionally does global_config.output.output_dir = resolve_output_dir(args.output_dir), which fires even when the flag was never passed. resolve_output_dir (backend/utils/app_home.py:83-100) never consults the config value ? with RALLY_APP_HOME unset it returns the argparse default "output" verbatim ? so a config.json output.output_dir can never take effect. db_path derives from the clobbered value (backend/main.py:1995-1997). The only production entry is main() ? create_app (main.py:2184) ? uvicorn.run (main.py:2185); there is no module-level app, so every server/CLI start hits the clobber. The knob is documented at backend/config/models.py:274-276. Not fixed: git log --oneline -15 -- backend/main.py tops out at e410136 (#413), none touching this. Split-state is concrete: backend/config/__init__.py:41-42 (default_db_path, used by run_eval.py/run_regression.py) builds the DB path directly from config.json's output.output_dir with no clobber ? so with the knob set, the server and the eval tooling resolve DIFFERENT DB locations. Not owner-gated (local config/CLI startup only).
222
223   ### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer
224   **upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live under RALLY_APP_HOME**
225   - anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness
226   - evidence: uploads.py:50 `upload_dir = getattr(sec, 'upload_dir', None) or 'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no resolve_output_dir/app_path is ever applied to it (only output.output_dir is resolved, main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.
227   - why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).
228   - proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg (relative ? app-home-rooted; absolute passes through), mirroring output_dir.
229   - verifier notes: Sharper failure scenario: on macOS the .command runs with Terminal's CWD=$HOME, and any `indexer --app` shell launch (advertised in install.py:243) uses the caller's CWD ? uploads land in e.g. ~/output/uploads while DB/reels live under ~/Library/Application Support/Khelsutra/output; unwritable CWD ? PermissionError at uploads.py:89 ? 500 on /api/upload/init. Caveat: Windows double-click of KhelSutra.bat coincidentally works (the .bat lives in app_home, install.py:175, and cmd sets CWD to the .bat dir). Extra impact the finder missed: tools/cleanup_caches.py:403 sweeps <output-dir>/uploads (app-home-rooted post-P0a) and the uninstall manifest user_data (install.py:117-121) lists only app-home paths, so stray multi-GB uploads are never swept and survive uninstall --purge. Fix as proposed is sane: resolve_output_dir() in _upload_cfg is byte-identical with RALLY_APP_HOME unset (app_home.py:94-95) and existing tests use absolute tmp paths (test_upload_download.py:31) that pass through.

6. assistant (2026-07-02T04:46:01.270Z)

Now let me read the repo's CLAUDE.md and CODE_MAP.

7. user (2026-07-02T04:46:02.190Z)

```
# CLAUDE.md ? project memory / agent handoff

Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
repos: `sports-data-collector`, a VLC-based `rally-annotator`.

## Read these first
- **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
  contracts (the cold-start map; read before touching code).
- **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
  live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
  disagree (licensing, Gemma-4 stance, base model, conversational-QA).
- **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
  `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
  (offline-segmenter narrative) and `docs/BACKLOG.md`.
- **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**
  `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
  (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
  "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`
(A/B + shadow
  eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
  (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
- **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
  **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move
  **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
  archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
  note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
  lifecycle register in **`docs/DOC_STATUS.md`**.

## Current state (June 2026)
- **Scaffolding complete**; strategy foundation + owned-model roadmap merged. **No
  platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
  sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
  validation, reliability, CI + test coverage) ? see git history.
- **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
  (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
  greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
  implementation without explicit owner approval.**

## Architecture in one breath
- **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
  / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
  `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
- `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
- Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
  user_version` migration ladder in `backend/infrastructure/database.py`.

```

```
47      ## Committed guardrails (non-negotiable)
48      - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49        training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50        code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51      - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52        is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
53        needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54        minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55      - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
56        (GDPR Art. 9).
57      - **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.
58      - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/`;
59        Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60        on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61        DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
62        `deploy/digitalocean/*` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64      ## Workflow conventions
65      - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
66      - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67      - Never push to the default branch directly; descriptive commit messages.
68      - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69      - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link
breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
70      - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
71        who has the golden corpus + labelled videos; verify pure-logic paths locally.
72
```

8. user (2026-07-02T04:46:04.166Z)

```
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
```

9. user (2026-07-02T04:46:12.828Z)

```
Already up to date.
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

10. user (2026-07-02T04:46:13.006Z)

```
No matches found
```

11. user (2026-07-02T04:46:17.234Z)

```
1       # CODE_MAP.md
2
3       **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5       **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
(this revision follows #40).
6
7       ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust
git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no
FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via
`segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not
basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec`
defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;
`match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked
`eval_baselines/` and **exits 3 on a missing baseline**; the API validates
`video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`,
`test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`,
`test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`,
`test_import_guard`, `test_cvat_robustness`).
8
9       **LEGEND**
10      - `@path` = file (always repo-relative)
11      - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
12      - `->` = dataflow / call / "produces"
13      - `§` = data contract (canonical shape)
14      - `?` = gotcha / footgun
15      - `?` = registry / plugin extension point
16      - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18      ---
19
20      ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
21
22      Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an
existing
23      **registry / extension point** (`?`) over a new top-level file.
24
25      | You're adding? | Put it in? | Notes |
26      |---|---|---|
27      | Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are **register a class in the relevant `?` registry**, not a new module tree. |
28      | **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29      | **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app
code) | **`scripts/`** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing
backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30      | **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31      | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
```

A test	`tests/test_*.py`	pytest auto-discovers; the full-suite lane has a **70% coverage floor**. Pure/offline + mocked (no GPU/WSL/network).
A doc (design, plan, living log)	`docs/`	Index it in `docs/README.md`. Only **terminal-state** (DONE/REJECTED) work moves to `docs/archives/`.
A throwaway repro / debug script	`scratch/`	**Gitignored** + excluded from ruff/pytest. Never imported by app code.
A model artifact (weights + manifest)	`artifacts/<name>/<ver>/`	ADR-008: **manifest committed** (provenance), **weights gitignored**.
Eval baseline / leaderboard JSON	`eval_baselines/`	Tracked (survives a clean checkout); the no-regression gate reads it.
Committed regression fixture (video-free golden)	`eval_baselines/fixtures/<slug>/`	Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0 (or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with **no video/GPU/DB**. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated).
Pipeline run outputs (reels, trajectories, DBs, frames)	`output/`	**Gitignored.** Per-video isolation is the tracked **`PER_VIDEO_WORKSPACE.md`** project (P0 helper shipped #264; supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`).

Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do not move):
`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run_all_tests.py` (CI: `ci.yml`) · `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) · `run_eval.py`
(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py` moved to
`scripts/doctor.py` (it shadowed the build system); invoke via `python scripts/doctor.py` or `indexer --setup`.
New standalone run-drivers go in **`scripts/`**, not the root.

---

1. PIPELINE

1A. Runtime: video file -> highlight reel
```
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-defaults, never fatal)
                                                  @backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig
       -> video_id = MD5(whole file)              @backend/utils/hashing.py::compute_video_id (single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)
       -> validate @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
            FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/pipeline/validators/__init__.py)
            [validation FAIL -> persist status="failed", return early; report STILL emitted in finally]
       -> db.add_video(video_id, filepath, status, [r.dict()]) @backend/infrastructure/database.py
       -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
       -> seg = segmenter_registry.get(seg_name)(db, global_config)   ? @backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
       -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
            [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]
       -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs ? even on validation failure / exception; NEVER raises; grafts response["reports"])
       -> select segment ids -> [(start,end)] (+ stitching padding)
       -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name) @backend/pipeline/stitcher/compiler.py
```

```
            per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
    ```
    Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
    ```
    P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
     -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted
via db.add_candidate_segment]
     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt)   ? provider_registry
     -> P4 db.add_play_segment + db.link_candidate_to_segment
    ```
    Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
    Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).

    ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
    ```
    WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
     -> WasbRunner.healthcheck()  (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
     -> WasbRunner.run_predict(video, out_dir)
          stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
          -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py  (shared, re-exported by WasbRunner)
     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING
BRAIN]
    ```
    (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)

    ### 1C. Calibration / eval flow (NO pipeline run unless told)
    ```
    GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
    DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage
? {candidates, final}
      -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
      -> harness.evaluate -> EvalReport -> report_to_dict
           -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
           -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py
[CI gate]
    Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate}  @backend/eval/calibration.py
    Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier)  @backend/eval/{calibrate_local,distill_local}.py
    Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
    Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies}  @backend/eval/gemini_refine.py
    Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
    ```
    Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py`
(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@scripts/nightly_regression.py` (nightly CACHED-trajectory CPU rally-quality tripwire, exit
1=regression/4=stale), `@scripts/nightly_reelcount.py` (nightly FULL-pipeline reel-count +
quality + cost on a GCP GPU VM, emailed), `@backend/eval/calibrate_wasb.py` +
```
```

`@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
106
107   ---
108
109   ## 2. SUBSYSTEMS
110
111   ### 2.0 Orchestration & config
112   - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
113   - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
114   - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx (path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202 + job_id**. `::_drain_due_jobs` / `::_run_claimed_job` (worker loop EXTRACTED to `@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via `db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling). `::_execute_processing` = the former sync body (hash -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in `result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
115   - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in `@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`; `uploads.py::_upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the startup config, and a lazy `import backend.main` avoids an import cycle) ? `uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`uploads.py::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight` `@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
116   - **STUDIO / STORAGE endpoints (P1/P2/P3/P4/P6/P8, 2026-07-01):** `::capabilities` `@GET /api/capabilities` ? honest box probe; now includes a **`storage` block** (`storage_capabilities` in `@backend/storage/capabilities.py`: local/s3/gcs/gdrive `{id,kind,label,usable,free_gb}`, secret-free) alongside segmenters/ffmpeg/gpu/deployment. `::list_videos` `@GET /api/videos` (the "my videos" list, P6). `::get_source` `@GET /api/source/{id}` ? Range-capable FileResponse serving a **frame-identical annotation proxy** (D4), source fallback, background-warmed. `::post_annotations` `@POST /api/annotations` ? writes the verbatim annotator CSV to `output/labels/<video_id>.rallies.csv` (P8). `::create_asset` `@POST /api/assets` (**P4**, `{uploaded_path?|source_uri?, medium_uri, sport?}`) ? stores a video into a user-chosen medium (MD5-keyed `storage.put` for an upload?remote medium; register-in-place for local; at-rest URI = no copy), registers it (video_id=MD5) so it shows in `/api/videos`; **self-host posture only** (403 when `security.allowed_video_root` set); edge-auth mirrors `/api/process`. Cost ledger: `::get_job_cost` `@GET /api/jobs/{id}/cost`, `::get_video_cost` `@GET /api/videos/{id}/cost`. **Free-space guard (P2):** `@backend/utils/freespace.py::require_free_bytes` -> HTTP **507** in `uploads.py::upload_init` +

the `/api/process` remote fetch-to-scratch; the WSL frame-extraction guard now **BLOCKS**
(`detectors/base.py::_wsl_frame_stage_shortfall`, ~15× source).
117   - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly.
`::ProcessRequest`/`::QueryRequest`/`::CompileRequest`/`::AnnotationsRequest`/`::AssetRequest`
pydantic bodies.
118   - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
119   - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `'motion'`.
120   - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive
`input()` (blocks automation), skips validation. Config via `backend.config.load_config`
(typed); segmenter, padding, and db path all read from the model (no literal fallbacks).

12. user (2026-07-02T04:46:23.637Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13
14      from __future__ import annotations
15
16      import os
17      import tempfile
18      from typing import Any, Dict, Iterator, Optional
19
20      from backend.storage.base import StorageBackend
21      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
22
23      __all__ = [
24          "StorageBackend",
25          "StorageRef",
26          "StorageStat",
27          "parse_uri",
28          "resolve_input_uri",
29          "get_storage",
30          "fetch",
31          "put",
32          "list_uris",
33          "acquire_local_input",
34          "input_is_local",
35          "resolve_input_ref",
36      ]
37
38
39      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
40          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
41          if storage_config is None:
```

```
            return {}
        if hasattr(storage_config, "model_dump"):
            return storage_config.model_dump()  # type: ignore[no-any-return]
        return dict(storage_config)


    def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
        """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
        config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
        callers at the API/CLI boundary catch that and return a clean client error (never a
500).

        Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
        ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
        sc = _storage_dict(storage_config)
        uri = resolve_input_uri(
            raw,
            input_backend=sc.get("input_backend", "local"),
            input_root=sc.get("input_root", ""),
        )
        return parse_uri(uri)


    def input_is_local(raw: str, storage_config: Any = None) -> bool:
        """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
        plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
        that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
        ``resolve_input_ref``)."""
        return resolve_input_ref(raw, storage_config).backend == "local"


    def acquire_local_input(
        raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
    ) -> str:
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
        regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
        worker's ``cmd_infer`` already ships."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
            return (
                ref.key
            )  # identity ? no temp, no copy, byte-identical to passing the path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)


    def _backend_class(name: str):
```

```
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage

            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage

            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage

            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage

            return GDriveStorage
        if name == "gdrive-api":
            from backend.storage.gdrive_api import GDriveApiStorage

            return GDriveApiStorage
        raise ValueError(
            f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
        )


    def get_storage(
        backend: Optional[str] = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        **overrides,
    ) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
        # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
        # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
        settings: Dict[str, Any] = dict(
            cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
        )
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(
        uri: str,
        dst: "str | os.PathLike[str] | None" = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        overwrite: bool = False,
    ) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(
            ref, dst, overwrite=overwrite
```

```
153         )
154 
155 
156     def put(
157         local_path: "str | os.PathLike[str]",
158         uri: str,
159         *,
160         config: Optional[Dict[str, Any]] = None,
161         content_type: Optional[str] = None,
162     ) -> str:
163         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
164         ref = parse_uri(uri)
165         get_storage(ref.backend, config=config).put(
166             local_path, ref, content_type=content_type
167         )
168         return uri
169 
170 
171     def list_uris(
172         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
173     ) -> Iterator[str]:
174         """Yield the URI of every object under ``prefix_uri``."""
175         ref = parse_uri(prefix_uri)
176         for r in get_storage(ref.backend, config=config).list(ref):
177             yield r.uri
178 
```

13. user (2026-07-02T04:46:31.298Z)

```
backend\config\models.py:378:    # scratch dir before processing (see
``backend.storage.acquire_local_input``).
backend\main.py:646:        local = storage.acquire_local_input(rec["filepath"], getattr(cfg,
"storage", None))
backend\main.py:791:            local_working = storage.acquire_local_input(
backend\main.py:1168:    local_video = storage.acquire_local_input(
backend\main.py:1683:        local_src = storage.acquire_local_input(
backend\main.py:2056:        local_video = storage.acquire_local_input(args.video_path,
storage_cfg)
backend\storage\__init__.py:33:    "acquire_local_input",
backend\storage\__init__.py:72:def acquire_local_input(
docs\PACKAGING_PLAN.md:154:| **CUJ-2** In-UI `gs://` bucket ingest | DONE |
`acquire_local_input` + gcs backend already accept `gs://`; preserved when the user opts into a
cloud storage backend (+ ADC). |
docs\COMPUTE_DECOUPLED_SERVING\README.md:161:- 2026-06-21 ? **M2 landed**
([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)):
`input_backend`/`input_root` + URI-aware main API/CLI input via
`resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.
tests\test_api_cloud_dispatch.py:114:        m.storage, "acquire_local_input", lambda path,
scfg: str(fake_local)
tests\test_api_cloud_dispatch.py:381:        m.storage, "acquire_local_input", lambda path,
scfg: str(fake_local)
tests\test_api_cloud_dispatch.py:556:        m.storage, "acquire_local_input", lambda path,
scfg: str(fake_local)
tests\test_input_source.py:2:acquire_local_input.
tests\test_input_source.py:19:from backend.storage import acquire_local_input, input_is_local,
resolve_input_ref
tests\test_input_source.py:124:#
------------------------------------------------------------- acquire_local_input
tests\test_input_source.py:125:def test_acquire_local_input_local_identity(tmp_path):
tests\test_input_source.py:129:    assert acquire_local_input(str(p)) == str(p)
tests\test_input_source.py:132:def test_acquire_local_input_relative_local_identity():
tests\test_input_source.py:133:    assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
```

```
tests\test_input_source.py:149:def
test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
tests\test_input_source.py:152:    out = acquire_local_input(
tests\test_input_source.py:162:def
test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
tests\test_input_source.py:165:    acquire_local_input("x.mp4", {"input_root":
"gs://bucket/videos"})
tests\test_input_source.py:169:def
test_acquire_local_input_never_fetches_for_local(monkeypatch):
tests\test_input_source.py:174:    assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
tests\test_input_source.py:176:        acquire_local_input("clip.mp4", {"input_backend":
"local", "input_root": ""})
```

14. user (2026-07-02T04:46:32.234Z)

```
190             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
191             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
192             LOG.info(
193                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
194             )
195             return ComputeResult(
196                 returncode=rc,
197                 outputs_uri=outputs_uri,
198                 video_id=video_id,
199                 report=report,
200                 execution=str(execution or ""),
201             )
202
203         # --- storage-backed completion signal (bucket-poll) ---
204         def _read_marker(self, done_uri: str) -> Optional[dict]:
205             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206             ref = parse_uri(done_uri)
207             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208                 return None
209             return (
210                 self._read_json(done_uri) or {}
211             )  # empty dict still terminates the poll (present == done)
212
213         def _read_json(self, uri: str) -> dict:
214             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
215             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
216             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
217             try:
218                 ref = parse_uri(uri)
219                 dst = os.path.join(
220                     tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
221                 )
222                 local = fetch(uri, dst, config=self._storage_config)
223                 with open(local, "r", encoding="utf-8") as f:
224                     data: Any = json.load(f)
225                 return data if isinstance(data, dict) else {}
226             except Exception:
227                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
228                 return {}
229
```

15. user (2026-07-02T04:46:39.074Z)

```
620                 _proxy_inflight.discard(video_id)
621
622
```

```
623     @app.get("/api/source/{video_id}")
624     def get_source(video_id: str, request: Request):
625         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
Serves a
626         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
still keyed
627         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
628         background for next time ? non-blocking, so the first load is never delayed. Proxy
warming is
629         LOCAL-source only (remote-source proxying is a follow-up)."""
630         rec = request.app.state.db.get_video(video_id)
631         if not rec or not rec.get("filepath"):
632             raise HTTPException(status_code=404, detail="Video not found.")
633         cfg = request.app.state.config
634
635         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
636         try:
637             proxy_name = safe_output_filename(f"{video_id}.mp4")
638             proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
_proxies_dir(cfg))
639         except ValueError:
640             proxy_path = ""
641         if proxy_path and os.path.isfile(proxy_path):
642             return FileResponse(proxy_path, media_type="video/mp4")
643
644         # 2) Otherwise resolve + serve the source.
645         try:
646             local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage",
None))
647             is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage",
None))
648         except ValueError as e:  # unsupported scheme etc. ? a clean client error, never a
500
649             raise HTTPException(status_code=400, detail=str(e)) from e
650         except Exception as e:  # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
651             raise HTTPException(status_code=502, detail=f"Could not resolve the source
video: {e}") from e
652         if not os.path.isfile(local):
653             raise HTTPException(status_code=404, detail="Source video file not found.")
654
655         # 3) Warm a proxy in the background for next time (local source only).
656         if is_local:
657             threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
658
659         media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
660         return FileResponse(local, media_type=media)
661
662
663     @app.post("/api/annotations")
664     def post_annotations(req: AnnotationsRequest, request: Request):
665         """Persist finished rally labels as the annotator CSV (P8), written verbatim to
666         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
``video_id`` ? so it joins
667         the video by content hash exactly as the training pipeline expects. Byte-compatible
with the VLC
668         plugin + browser extension. A later step (owner-gated) promotes labels into the
durable corpus.
669
670         Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
validated as a bare
671         filename so an untrusted value can't escape the labels dir (arbitrary write)."""
672         try:
```

```
            edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        cfg = request.app.state.config
        labels_dir = os.path.join(output_base(cfg), "labels")
        try:
            name = safe_output_filename(f"{req.video_id}.rallies.csv")
        except ValueError as e:
            raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
        os.makedirs(labels_dir, exist_ok=True)
        path = os.path.join(labels_dir, name)
        try:
            path = resolve_under_root(path, labels_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        with open(path, "w", encoding="utf-8", newline="") as f:
```

16. user (2026-07-02T04:46:39.920Z)

```
                validate_input_path(req.uploaded_path, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
                raise HTTPException(
                    status_code=404,
                    detail=f"uploaded file not found at '{req.uploaded_path}'",
                )
            local_working = src_ref.key
        else:
            try:
                validate_input_path(req.source_uri, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
scratch (P2).
            if src_ref.backend != "local":
                need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
                require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
            try:
                local_working = storage.acquire_local_input(
                    req.source_uri, getattr(cfg, "storage", None)
                )
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            except Exception as e:  # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
                raise HTTPException(
                    status_code=502, detail=f"could not resolve source_uri: {e}"
                ) from e
            if not os.path.isfile(local_working):
                raise HTTPException(
                    status_code=404, detail="source_uri did not resolve to a readable file."
                )

        video_id = compute_video_id(local_working)
        ext = os.path.splitext(local_working)[1] or ".mp4"

        # 2) Decide the durable location + copy only when needed.
        copied = False
        if req.source_uri:
            stored_uri = req.source_uri  # already at rest in its medium ? no copy (D14b)
            medium_backend = src_ref.backend
```

```
        else:
            try:
                medium_ref = storage.parse_uri(req.medium_uri)
            except ValueError as e:
                raise HTTPException(
                    status_code=400, detail=f"invalid medium_uri: {e}"
                ) from e
            medium_backend = medium_ref.backend
            if medium_ref.backend == "local":
                # "This computer" ? the upload already lives here; it IS the durable copy
(no duplicate).
                stored_uri = os.path.abspath(local_working)
            else:
                dest = _asset_dest_uri(req.medium_uri, video_id, ext)
                # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud bucket is
warn-only.
                local_dir = _medium_local_dir(medium_ref, storage_settings)
                if local_dir is not None:
                    require_free_bytes(
                        local_dir, os.path.getsize(local_working), stage="store"
                    )
                else:
                    logger.info(
                        "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota warn
only).",
                        medium_ref.backend,
                    )
                try:
                    stored_uri = storage.put(local_working, dest, config=storage_settings)
                    copied = True
                except ValueError as e:
                    raise HTTPException(status_code=400, detail=str(e)) from e
                except Exception as e:  # noqa: BLE001 ? a put failure is surfaced, never
silently dropped
                    logger.error(
                        "[ASSETS] put failed video_id=%s dest=%s: %s",
                        video_id,
                        dest,
                        e,
                        exc_info=True,
                    )
                    raise HTTPException(
                        status_code=502, detail=f"could not store into the medium: {e}"
                    ) from e

        # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
videos").
        if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
            raise HTTPException(status_code=500, detail="failed to register the asset.")
        logger.info(
            "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
            video_id,
            medium_backend,
            stored_uri,
            copied,
            owner_id,
        )
        return JSONResponse(
            status_code=201,
            content={
                "video_id": video_id,
                "medium": medium_backend,
                "stored_uri": stored_uri,
                "copied": copied,
                "sport": req.sport,
```

```
873                 "status": "stored",
874             },
875         )
876
877
878     def _ingest_remote_segments(
879         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
880     ) -> int:
881         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
882         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
883         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
884         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
885         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
886         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
887         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
888         and prunes any partial rows (destroy-AFTER-confirm)."""
889         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
```

17. user (2026-07-02T04:46:46.438Z)

```
1130
1131
1132    def _execute_processing(
1133         config=None, db=None, req=None, progress=None
1134    ) -> Dict[str, Any]:
1135         """The full hash ? validate ? segment ? report pipeline for one video.
1136
1137         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
1138         module-level ``global_config``/``db_instance`` (backward-compatible with existing
1139         tests that pass only ``req``). Route handlers always pass explicit params; older
1140         test code that calls ``_execute_processing(req)`` still works.
1141
1142         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
1143         Raises on unexpected internal errors ? the caller records those as a job failure
1144         """
1145         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
1146         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
1147         if config is not None and not isinstance(config, AppConfig):
1148             # Old signature: _execute_processing(req, progress=None)
1149             if req is None and db is not None and isinstance(db, ProcessRequest):
1150                 req = db
1151                 db = None
1152             elif req is None and isinstance(config, ProcessRequest):
1153                 req = config
1154                 config = None
1155         # Resolve config/db from module globals when not passed explicitly
1156         if config is None:
1157             config = global_config
1158         if db is None:
1159             db = db_instance
1160
1161         notify = progress or (lambda stage: None)
1162
1163         notify("hashing")
1164         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
```

```
1165        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
1166        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
1167        # consumers (hash / validators / segmenter) use the resolved local path.
1168        local_video = storage.acquire_local_input(
1169            req.video_path, getattr(config, "storage", None)
1170        )
1171        video_id = compute_video_id(local_video)
1172        was_reingest = db.get_video(video_id) is not None
1173
1174        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
1175        notify("validating")
1176        logger.info(f"Running validations for {req.video_path}...")
1177        val_results, val_context = registry.run_all_with_context(local_video, config)
1178        failures = [r for r in val_results if not r.passed]
1179
1180        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181        default_seg = getattr(
1182            getattr(config, "indexing", None), "default_segmenter", "motion"
1183        )
1184        seg_name = req.segmenter or default_seg
1185        segmenter_actual = None
1186        segmentation_ran = False
1187        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190        compute_actual: Optional[str] = None
1191        response: Optional[Dict[str, Any]] = None
1192        try:
1193            if failures:
1194                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1195                # status; it no longer destroys the video's prior good segments.
1196                db.add_video(
1197                    video_id,
1198                    req.video_path,
1199                    "failed",
1200                    [r.model_dump() for r in val_results],
1201                )
1202                response = {
1203                    "video_id": video_id,
1204                    "status": "failed",
1205                    "message": "Video failed validation checks.",
1206                    "results": [r.model_dump() for r in val_results],
1207                }
1208                return response
1209
1210            # 2. Run segmenter & indexer
1211            logger.info("[API] Video passed checks. Segmenting and indexing...")
1212            db.add_video(
1213                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
1214            )
1215
1216            segmenter_cls = segmenter_registry.get(seg_name)
1217            if not segmenter_cls:
1218                # Submit-time validation already 400s unknown names; this is the defensive
1219                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
1220                available = list(segmenter_registry.list_available().keys())
1221                response = {
1222                    "video_id": video_id,
```

```
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db.segment_watermarks(video_id)
            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
            # they're dropped on the cloud path in this slice ? a documented follow-up.)
            remote = _maybe_dispatch_remote(
                config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
            )
            if remote is not None:
                response, segmentation_ran = (
                    remote  # `ran` = did the cloud job actually execute (report)
                )
                return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
            compute_actual = "in_process"
            segmenter = segmenter_cls(db, config)
            try:
                result = segmenter.process_video(
                    video_id, local_video, req.player_pool, req.max_frames
                )
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
```

```
            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault(
                    "compute",
                    {
                        "path": compute_actual or "not_run",
                        "requested": getattr(req, "compute", None),
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = emit_indexer_report(
                db=db,
                video_id=video_id,
                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
                # still preserved in DB provenance via db.add_video.
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=req.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

```

```
1333
1334    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1335        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1336
1337        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
1338        missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
1339        to a no-op rather than blocking a job it cannot measure."""
1340        try:
1341            sc = (
1342                storage_config.model_dump()
1343                if hasattr(storage_config, "model_dump")
1344                else (storage_config or {})
1345            )
1346            backend = storage.get_storage(input_ref.backend, config=sc)
1347            return int(backend.stat(input_ref).size)
1348        except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must
not block
1349            logger.debug(
1350                "free-space: could not stat remote source %s: %s", input_ref.uri, e
1351            )
1352            return None
1353
1354
1355    @app.post("/api/process", status_code=202)
1356    def process_video(req: ProcessRequest, request: Request):
1357        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1358
1359        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
```

18. user (2026-07-02T04:46:53.566Z)

```
1640            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1641        )
1642
1643
1644    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1645        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1646        and return the {status, filepath, download_url} result the SPA polls for. Returns a
1647        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1648        so the SPA always gets a clean verdict instead of a worker-crash error."""
1649        notify = progress or (lambda stage: None)
1650        params = json.loads(job.get("params") or "{}")
1651        video_id = job.get("video_id") or ""
1652        segment_ids = params.get("segment_ids") or []
1653        locale = params.get("locale")
1654
1655        notify("resolving")
1656        try:
1657            video, kept, out_name = _resolve_compile(
1658                db, cfg, video_id, segment_ids, params.get("output_filename", "")
1659            )
1660        except _CompileError as e:
1661            return {"status": "failed", "message": e.detail, "video_id": video_id}
1662
1663        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1664        output_dir = (
1665            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1666        )
1667        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1668        # unknown/absent locale keeps the configured default unchanged.
```

```
        localized_watermark = localize_watermark(stitching.watermark, locale)
        compiler = VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        notify("stitching")
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
            local_src = storage.acquire_local_input(
                video["filepath"], getattr(cfg, "storage", None)
            )
            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }

        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        notify("finished")
        return {
            "status": "success",
            "filepath": out_path,
            "download_url": download_url,
            "video_id": video_id,
        }


    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
```

```
1729        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
1730        # Lazy import: cv2 is NOT installed in the LIGHT tier (torch-free default)
1731        # and this --debug-clip path is CLI-only, not on the served API path.
1732        import cv2
1733
1734        cap = cv2.VideoCapture(video_path)
1735        if not cap.isOpened():
1736            print("[FAIL] OpenCV could not open video.")
1737            return
1738
1739        fps = cap.get(cv2.CAP_PROP_FPS)
1740        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
1741
1742        start_frame = max(0, int((timestamp - 3.0) * fps))
1743        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
1744
1745        # Measure frame-by-frame changes in sample region
1746        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
1747        prev_gray = None
1748        motions = []
1749
1750        for _ in range(start_frame, end_frame):
1751            ret, frame = cap.read()
1752            if not ret:
1753                break
1754            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
1755            gray = cv2.GaussianBlur(gray, (21, 21), 0)
1756
1757            if prev_gray is not None:
1758                diff = cv2.absdiff(prev_gray, gray)
1759                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
1760                motion_ratio = cv2.countNonZero(thresh) / (
1761                    thresh.shape[0] * thresh.shape[1]
1762                )
1763                motions.append(motion_ratio)
1764            prev_gray = gray
1765
1766        cap.release()
1767
1768        if motions:
1769            avg_motion = sum(motions) / len(motions)
```

19. user (2026-07-02T04:46:59.420Z)

```
1960                return
1961            run_cli_diagnose_clip(args.video_path, args.debug_clip)
1962            return
1963
1964        # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly
`--app`
1965        # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1966        # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
1967        # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1968        # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1969        # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1970        # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1971        if args.app:
1972            seeded_path, seeded = write_light_default_config()
1973            if seeded:
```

```
                print(
                    f"[APP] First run: seeded a truly-local default config at {seeded_path}
"
                    f"(detector 'motion' ? offline, no API key or GPU needed). "
                    f"Use the in-app setup to switch to Gemini."
                )

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig
        _enforce_startup_or_exit(
            global_config
        )  # M5: fail fast on an incoherent ACTIVE profile

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
        # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
        global_config.output.output_dir = resolve_output_dir(args.output_dir)
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(
            global_config.output.output_dir, global_config.output.db_name
        )

        # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
        # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
        # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
        # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
        global _instance_lock
        try:
            _instance_lock = acquire_lock(db_path + ".lock")
        except OSError as exc:
            # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
            # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
            # legitimate single instance).
            logger.warning(
                "[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
                "WITHOUT the second-instance guard.",
                db_path,
                exc,
            )
            _instance_lock = None
        else:
            if _instance_lock is None:
                print(
                    f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
                    f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
                    f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
                )
                sys.exit(1)

        db_instance = Database(db_path)
```

```
2027
2028        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
2029        # the executor is in-process. Mark them failed so pollers see a terminal state.
2030        swept = db_instance.fail_stale_jobs()
2031        if swept:
2032            logger.warning(
2033                f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed."
2034            )
2035
2036        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
2037        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
2038        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
2039        _maybe_sweep_gemini_orphans(global_config)
2040
2041        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
2042        if args.video_path and not args.server and not args.app:
2043            print(f"[CLI] Processing video file: {args.video_path}")
2044            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
2045            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
2046            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
2047            storage_cfg = getattr(global_config, "storage", None)
2048            try:
2049                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
2050            except ValueError as e:
2051                print(f"[FAIL] {e}")
2052                return
2053            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
2054                print(f"[FAIL] Video file not found: {args.video_path}")
2055                return
2056            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
2057
2058            video_id = compute_video_id(local_video)
2059            was_reingest = db_instance.get_video(video_id) is not None
2060
2061            # Validate (with-context variant surfaces ffprobe_meta for the report)
2062            print("[CLI] Validating...")
2063            val_results, val_context = registry.run_all_with_context(
2064                local_video, global_config
2065            )
2066            failures = [r for r in val_results if not r.passed]
2067
2068            # Save video status
2069            status = "failed" if failures else "processed"
2070            db_instance.add_video(
2071                video_id, args.video_path, status, [r.model_dump() for r in val_results]
2072            )
2073
2074            seg_name = args.segmenter or getattr(
2075                getattr(global_config, "indexing", None), "default_segmenter", "motion"
2076            )
2077            segmenter_actual = None
2078            segmentation_ran = False
2079            try:
2080                print("\n" + "=" * 40)
2081                print("VALIDATION SUMMARY")
2082                print("=" * 40)
2083                for r in val_results:
2084                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
```

```
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("=" * 40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(
                    video_id, local_video, max_frames=args.max_frames
                )
                segmentation_ran = True

                if isinstance(result, Failure):
                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                    print("[FAIL] Indexing halted for video.")
                    segments = []
                else:
                    segments = result.value if hasattr(result, "value") else result
                    print(
                        f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}"
                    )
                # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
                _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
            finally:
                # Always emit the per-video observability report. Never raises.
                paths = emit_indexer_report(
                    db=db_instance,
                    video_id=video_id,
                    # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI would
write a
                    # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source URI
is kept
                    # in DB provenance via db.add_video.
                    video_path=local_video,
                    config=global_config,
                    val_results=val_results,
                    val_context=val_context,
                    segmenter_requested=args.segmenter,
                    segmenter_actual=segmenter_actual,
                    was_reingest=was_reingest,
                    segmentation_ran=segmentation_ran,
                )
                if paths:
                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
            return

        # Start Local Server Mode
        if args.app or args.server or not args.video_path:
            if args.app:
                # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
```

```
                # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
                bind_host = "127.0.0.1"
                port = _resolve_app_port(bind_host, args.port)
                if port != args.port:
                    print(f"[APP] Port {args.port} is busy ? using {port} instead.")
                url = f"http://{bind_host}:{port}/"
                # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
                # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
                # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
                print(
                    f"[APP] Starting KhelSutra ? opening {url} in your browser "
                    f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop."
                )
                threading.Thread(
                    target=_open_browser_when_ready,
                    args=(url, url + "api/health"),
                    daemon=True,
                ).start()
            else:
                bind_host = _resolve_bind_host(
                    global_config, args.host
                )  # P0c: config/profile-driven (default 127.0.0.1)
                port = args.port
                print(
                    f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}..."
                )
                if bind_host not in ("127.0.0.1", "localhost", "::1"):
                    print(
                        f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
                        f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
                        f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
                        f"loopback-only and will 400 forwarded requests otherwise)."
                    )
            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
            _get_executor().submit(_drain_due_jobs, global_config, db_instance)
            # Stamp shared state on the app so route handlers access it via
request.app.state
            # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
            create_app(global_config, db_instance)
            uvicorn.run(app, host=bind_host, port=port)


    if __name__ == "__main__":
        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
        # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
        # module object. If `main()` runs here it configures `__main__`'s globals, leaving
        # `backend.main.db_instance` = None; the async-job drain then crashes silently on
        # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
        # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
        # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
```

```
fire.)
2196        import backend.main as _bm
2197
2198        _bm.main()
2199
```

20. user (2026-07-02T04:47:06.008Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18
19      import logging
20      import threading
21      import traceback
22      from concurrent.futures import ThreadPoolExecutor
23      from typing import Any, Dict, Optional
24
25      logger = logging.getLogger(__name__)
26
27      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
28      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
29      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
30      # already parallelize their AI handoff internally via their own pools.
31      #
32      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
33      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
34      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
35      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
36      #   - detector timeout_sec kills hung WSL/GPU calls
37      #   - Cloud Run poll has a max_wait_sec bound
38      # A future slice should add a per-job wall-clock timeout (the executor itself
39      # cannot enforce timeouts on the Thread).
40      _job_executor: Optional[ThreadPoolExecutor] = None
41
42
43      def _get_executor() -> ThreadPoolExecutor:
44          """Lazy module-global executor; tests monkeypatch this for inline execution."""
45          global _job_executor
46          if _job_executor is None:
47              _job_executor = ThreadPoolExecutor(
48                  max_workers=1, thread_name_prefix="jobworker"
49              )
50          return _job_executor
51
52
53      class JobWaiting(Exception):
54          """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
```

```
55          WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
56          NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
57          releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
58          is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
59
60          The wiring is additive: the production segmenter path never raises this today (zero
61          behaviour change), so a job runs exactly as before. The hook is what a later slice
62          (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
63          """
64
65          def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
66              super().__init__(f"job parked for {delay_sec:.0f}s")
67              self.delay_sec = max(0.0, float(delay_sec))
68              self.progress = progress
69
70
71      def _job_to_request(job: Dict[str, Any]):
72          """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
73          re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
74          the original submit. The row stores exactly the ProcessRequest fields."""
75          import backend.main as _main
76
77          return _main.ProcessRequest(
78              video_path=job["video_path"],
79              player_pool=job.get("player_pool"),
80              max_frames=job.get("max_frames"),
81              segmenter=job.get("segmenter"),
82              # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
83              # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
84              # the job row for analytics). NULL ? server default = pre-picker behaviour.
85              compute=job.get("compute"),
86          )
87
88
89      def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
90          """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
91          terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
92
93          Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
94          the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
95          result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
96          job self-scheduled and will be re-claimed when `next_poll_at` is due.
97          """
98          import backend.main as _main
99
100         job_id = job["id"]
101         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
102         # original /api/process pipeline). Both record their terminal state the same way;
only process
103         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
104         is_compile = job.get("kind") == "compile"
105         try:
106             if is_compile:
107                 result = _main._execute_compile(
108                     config,
```

```
                    db,
                    job,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            else:
                req = _main._job_to_request(job)
                result = _main._execute_processing(
                    config,
                    db,
                    req,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            if result.get("video_id"):
                db.set_job_video_id(job_id, result["video_id"])
            db.mark_job_done(job_id, result)
            if not is_compile:
                _maybe_record_job_cost(
                    job, result, config, db
                )  # M8 cost ledger (default-OFF; process jobs only)
                _maybe_run_flywheel(
                    job, result, config
                )  # M11 data-flywheel (process jobs only)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _maybe_record_job_cost(
        job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
    ) -> None:
        """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
        ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
        (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
        persist it.

        ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
        ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
        FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
        + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
```

```
                owner_id=job.get("owner_id"),
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
        (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
        land), so this captures **nothing** in production. When activated it captures the
consented job's
        artifacts into the per-video workspace (? later labeling ? the golden training set).

        Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
        row (carries ``owner_id``); ``result`` is the pipeline output."""

        flywheel_cfg = getattr(config, "flywheel", None)
        if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
            return
        try:
            from backend import flywheel

            flywheel.run_for_job(job, result, config)
        except Exception as e:  # never wedge a completed job on the flywheel
            logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)


    def _drain_due_jobs(config: Any, db: Any) -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main

        ran = 0
        while True:
            job = db.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job, config, db)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
```

```
new
224         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
225         # on the next submit/drain. Best-effort ? never raises into the worker.
226         _main._schedule_next_drain_if_waiting(config, db)
227         return ran
228
229
230     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
231         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
232         self-scheduled job resumes with no further client submit. The drain itself re-checks
233         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
234         import backend.main as _main
235
236         try:
237             delay = db.seconds_until_next_due()
238         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
239             logger.error(f"Could not compute next-due delay: {e}")
240             return
241         if delay is None:
242             return  # nothing waiting
243         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
244         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
245
246
247     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
248         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
249         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
250         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
251         import backend.main as _main
252
253         timer = threading.Timer(
254             delay_sec,
255             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
256         )
257         timer.daemon = True
258         timer.start()
259
260
261     #: A parked job further out than this is re-checked by a capped wake rather than one
long
262     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
263     _MAX_DRAIN_WAKE_SEC = 60.0
264
```

21. user (2026-07-02T04:47:12.227Z)

```
backend\main.py:1106:        "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
backend\worker\__main__.py:209:    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-
worker-")
backend\worker\__main__.py:473:    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-
train-")
backend\compute\__init__.py:66:    job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
backend\compute\cloud_run.py:220:                tempfile.mkdtemp(prefix="cr-read-"), ref.name
or "obj.json"
backend\storage\__init__.py:89:    scratch = tempfile.mkdtemp(prefix="rally-input-",
dir=scratch_parent)
backend\eval\golden_real_fixtures.py:355:        tempfile.mkdtemp(prefix=f".{out_slug}.tmp.",
dir=str(fixtures_dir))
```

22. user (2026-07-02T04:47:20.284Z)

```
180                 "returncode": returncode,
181                 "segment_count": segment_count,
182                 "status": status,
183             }
184             local = os.path.join(scratch, "_done.json")
185             with open(local, "w", encoding="utf-8") as f:
186                 json.dump(marker, f)
187             storage.put(local, done_uri, config=storage_cfg)
188             logger.info("[worker] wrote completion marker -> %s", done_uri)
189         except Exception as e:  # never fail a good run because the marker push hiccuped
190             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
191
192
193     def cmd_infer(args: argparse.Namespace) -> int:
194         config = load_config(args.config) if args.config else load_config()
195         _apply_overrides(config, args.set)
196         if not _run_startup_checks(config):
197             return 2
198
199         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
200         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
201         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
202         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
203         compute = get_compute_backend(config)
204         if compute is not None:
205             return _dispatch_remote(compute, args)
206
207         storage_cfg = config.storage.model_dump()
208
209         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
210         os.makedirs(scratch, exist_ok=True)
211         config.output.output_dir = scratch
212
213         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
214         local_video = storage.fetch(
215             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
216         )
217         print(f"[worker] pulled {args.video_uri} -> {local_video}")
218
219         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
220         video_id = compute_video_id(local_video)
221
222         # 3. VALIDATE (same registry as the main CLI).
223         val_results, val_context = validator_registry.run_all_with_context(
224             local_video, config
225         )
226         failures = [r for r in val_results if not r.passed]
227         db = Database(os.path.join(scratch, config.output.db_name))
228         db.add_video(
229             video_id,
230             local_video,
231             "failed" if failures else "processed",
232             [r.model_dump() for r in val_results],
233         )
234
235         def _fail(rc: int) -> int:
236             # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
```

```
so a remote
237             # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until
its poll
238             # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
239             if getattr(args, "done_uri", None):
240                 _write_done_marker(
241                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
242                 )
243             return rc
244 
245         segments: List[dict] = []
246         segmenter_actual = None
247         segmentation_ran = False
248         status = "failed"
249         try:
250             if failures:
251                 print(
252                     "[worker] validation FAILED: "
253                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
254                 )
255                 return _fail(2)
256             seg_name = args.segmenter or config.indexing.default_segmenter
257             segmenter_cls = segmenter_registry.get(seg_name)
258             if not segmenter_cls:
259                 print(
260                     f"[worker] unknown segmenter: {seg_name!r} "
261                     f"(have {list(segmenter_registry.list_available())})"
262                 )
263                 return _fail(2)
264             segmenter_actual = seg_name
265             result = segmenter_cls(db, config).process_video(
266                 video_id, local_video, max_frames=args.max_frames
267             )
268             segmentation_ran = True
269             if isinstance(result, Failure):
270                 print(f"[worker] segmenter FAILED: {result.message}")
271                 return _fail(3)
272             segments = result.value if hasattr(result, "value") else result
273             status = "success"
274             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
275             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
276             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
277             if not segments:
278                 detector_impl = (
279                     os.environ.get("RALLY_DETECTOR_IMPL")
280                     or getattr(config.indexing, "detector_impl", None)
281                     or "auto"
282                 )
283                 logger.warning(
284                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
285                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
286                     "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
287                     "the WASB env produced no detections, or the input resolution differs
from the tuned "
288                     "proxy resolution. Re-run with -v for the native command + frame
counts.",
289                     video_id,
290                     segmenter_actual,
```

```
291                     detector_impl,
292                 )
293         finally:
294             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
295             emit_indexer_report(
296                 db=db,
297                 video_id=video_id,
298                 video_path=local_video,
299                 config=config,
```

23. user (2026-07-02T04:47:43.404Z)

```
backend\reporting\__init__.py:29:def emit_indexer_report(
backend\reporting\__init__.py:74:        output_dir = (config.get("report") or
{}).get("output_dir")
backend\reporting\__init__.py:75:        res = rec.write(output_dir=output_dir)
```

24. user (2026-07-02T04:47:47.608Z)

```
1       """Per-video observability reports for the indexer.
2
3       `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4       no-op and the pipeline runs exactly as before. The single entrypoint
5       `emit_indexer_report(...)` is designed to be called from a `finally` block and
6       NEVER raises ? emitting a report must not be able to break a processing run.
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, Dict, List, Optional
12
13      from . import run_signals  # no obsreport dependency ? always importable
14
15      try:  # soft dependency ? never hard-fail the pipeline on a reporting import
16          import obsreport  # noqa: F401
17
18          from . import harvest
19
20          OBSREPORT_AVAILABLE = True
21      except Exception:  # pragma: no cover - exercised only when obsreport is absent
22          OBSREPORT_AVAILABLE = False
23
24
25      def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
26          return bool((config or {}).get("report", {}).get("enabled", True))
27
28
29      def emit_indexer_report(
30          *,
31          db,
32          video_id: str,
33          video_path: Optional[str],
34          config: Optional[Dict[str, Any]],
35          val_results: List[Any],
36          val_context: Optional[Dict[str, Any]] = None,
37          segmenter_requested: Optional[str] = None,
38          segmenter_actual: Optional[str] = None,
39          was_reingest: bool = False,
40          segmentation_ran: bool = True,
41      ) -> Optional[Dict[str, str]]:
42          """Harvest + write the report. Returns the written file paths, or None if
43          reporting is unavailable/disabled/failed. Never raises."""
44          config = config or {}
45          # Accept either a plain dict or a typed config object (e.g.
```

```
backend.config.AppConfig).
46          # Normalize to a plain nested dict so harvest's dict access works uniformly.
47          if hasattr(config, "model_dump"):
48              try:
49                  config = config.model_dump(mode="json")
50              except Exception:
51                  config = {}
52          # Consume run signals unconditionally (even when reporting is off) so the
53          # process-local store never accumulates entries for finished runs.
54          signals = run_signals.pop(video_id)
55          if not OBSREPORT_AVAILABLE or not report_enabled(config):
56              return None
57          try:
58              play_segments = db.query_play_segments(video_id, {}) if db is not None else []
59              candidates = db.query_candidate_segments(video_id) if db is not None else []
60              rec = harvest.record_run(
61                  video_id=video_id,
62                  video_path=video_path,
63                  config=config,
64                  val_results=val_results,
65                  val_context=val_context,
66                  play_segments=play_segments,
67                  candidates=candidates,
68                  segmenter_requested=segmenter_requested,
69                  segmenter_actual=segmenter_actual,
70                  was_reingest=was_reingest,
71                  segmentation_ran=segmentation_ran,
72                  run_signals=signals,
73              )
74              output_dir = (config.get("report") or {}).get("output_dir")
75              res = rec.write(output_dir=output_dir)
76              return res.paths or None
77          except Exception:  # pragma: no cover - defensive; reporting must never break a run
78              return None
79
```

25. assistant (2026-07-02T04:49:11.430Z)

Now let me check the tests that monkeypatch `acquire_local_input`, the config models around storage, and `_maybe_sweep_gemini_orphans` for pattern reference.

26. user (2026-07-02T04:49:13.323Z)

```
80                  json.dump(
81                      {
82                          "video_id": self.video_id,
83                          "returncode": self.returncode,
84                          "segment_count": len(self.rows),
85                          "status": "success",
86                      },
87                      f,
88                  )
89              return "exec-1"
90
91
92      @pytest.fixture
93      def cloud_env(tmp_path, monkeypatch):
94          """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
95          identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
96          db = Database(str(tmp_path / "t.db"))  # noqa: F841
97          monkeypatch.setattr(m, "db_instance", db)
98          monkeypatch.setattr(m, "db_instance", db)
99          bucket = tmp_path / "bucket"
```

```
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": "cloud_run"},
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token. ``**kw``
        forwards the per-request ``target_override`` (item 3) so the cloud-picker path
resolves too."""
        real = compute_pkg.get_compute_backend
        monkeypatch.setattr(
            compute_pkg,
            "get_compute_backend",
            lambda config, **kw: real(
                config, run_client=fake, policy=_FAST, token="tok", **kw
            ),
        )


    def _meta(row):
        md = row.get("metadata")
        return json.loads(md) if isinstance(md, str) else (md or {})


    def test_cloud_dispatch_ingests_segments(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        rows = db.query_play_segments(out["video_id"], {})
        assert len(rows) == 2
        metas = [_meta(r) for r in rows]
        assert all(md["source"] == "cloud_run" for md in metas)
        assert {md.get("shot_count") for md in metas} == {5, 3}
        assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}


```

27. user (2026-07-02T04:49:14.240Z)

```
350         edge_auth_enabled: bool = False
351         cf_team_domain: str = (
352             ""  # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
353         )
354         cf_access_aud: str = ""  # the CF Access application AUD tag the token must carry
355         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
356         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
357         # OFF ? Cf-Access is a header, not a cookie.)
358
359
360     class StorageConfig(BaseModel):
361         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
362         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
363         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
364         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
365         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
366         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
367
368         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
369         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
370         output_root: str = ""
371         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
372         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
373         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
374         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
375         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
376         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
377         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
378         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
379         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
380         input_root: str = ""
381         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
382         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
383         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
384         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
385         # the legacy flat layout is used unchanged.
386         single_folder_per_video: bool = False
387         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
388         workspace_root: str = ""
389         # Cloud namespace prefix under the root so per-video folders sit tidily beside
390         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
391         workspace_prefix: str = "workspaces"
392         local: dict[str, Any] = Field(default_factory=dict)
393         s3: dict[str, Any] = Field(default_factory=dict)
394         gcs: dict[str, Any] = Field(default_factory=dict)
```

```
395         gdrive: dict[str, Any] = Field(default_factory=dict)
396         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
397         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
398         gdrive_api: dict[str, Any] = Field(default_factory=dict)
399
400
401     class DeploymentConfig(BaseModel):
402         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
403         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
404
405         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
406         behaves exactly as before this section existed. A profile is opt-in (``config.json``
407         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
408         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
409         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
410         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
411
412         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
413
414         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
415         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
416         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
417         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
418         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
419
420
421     class BillingConfig(BaseModel):
422         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
423
424         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
425         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
426         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
427         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
428         inserts a real, calibrated version and points ``pricebook_version`` at it.
429
```

28. user (2026-07-02T04:49:20.813Z)

```
1808:def _maybe_sweep_gemini_orphans(config) -> None:
1809-    """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed
run.
1810-    A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
1811-    a failed sweep is logged and swallowed. The Gemini client is built with an HTTP timeout
so
1812-    a hung connection can't stall startup indefinitely. Extracted for unit tests."""
1813-    try:
1814-        provider_name = getattr(config, "ai_provider", "")
1815-        if provider_name != "gemini":
1816-            return
1817-        api_key = os.environ.get("GEMINI_API_KEY")
1818-        if not api_key:
```

```
1819-            return
1820-        provider = provider_registry.get(
1821-            "gemini", api_key=api_key, model_name=config.ai_model
1822-        )
1823-        deleted = provider.cleanup_orphaned_files()
1824-        if deleted:
1825-            logger.info(
1826-                "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
1827-            )
1828-    except Exception as e:
1829-        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1830-
1831-
1832-def main():
1833-    parser = argparse.ArgumentParser(
1834-        description="Sports Highlight Indexer Orchestrator"
1835-    )
1836-    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1837-    parser.add_argument(
1838-        "--output-dir",
1839-        default="output",
1840-        help="Directory where processed clips/highlights are saved",
1841-    )
1842-    parser.add_argument(
1843-        "--setup",
1844-        action="store_true",
1845-        help="Launch dependency checker and setup utility",
1846-    )
1847-    parser.add_argument(
1848-        "--debug-clip",
```

29. user (2026-07-02T04:49:27.020Z)

```
1       """Input acquisition for COMPUTE_DECOUPLED_SERVING M2 ? resolve_input_uri /
input_is_local /
2       acquire_local_input.
3
4       The hard guarantee: the default config (input_backend='local', input_root='') is a byte-
for-byte
5       identity passthrough for every input path used today (no fetch, no copy). A bucket/Drive
source is
6       routed through storage.fetch to a local scratch file. All offline (the remote backend is
a
7       monkeypatched fetch ? no boto3/google-cloud-storage, no network).
8
9       OS note: os.path.isabs is platform-dependent, so the absolute-path cases use a POSIX-
style
10      '/data/...' (absolute on BOTH Windows and POSIX) and Windows backslash paths only with
input_root=''
11      (branch 5, OS-independent) ? the full suite runs cross-OS in CI.
12      """
13
14      import os
15
16      import pytest
17
18      from backend import storage
19      from backend.storage import acquire_local_input, input_is_local, resolve_input_ref
20      from backend.storage.ref import resolve_input_uri
21
22
23      # ------------------------------------------------------------- resolve_input_uri
24      @pytest.mark.parametrize(
25          "raw,ib,ir,expected",
26          [
```

```
            # 1. explicit scheme always wins (even over a non-local backend + root)
            ("gs://b/x.mp4", "local", "", "gs://b/x.mp4"),
            ("s3://b/x.mp4", "gcs", "gs://r", "s3://b/x.mp4"),
            ("local://abs/x.mp4", "gcs", "gs://r", "local://abs/x.mp4"),
            ("gdrive://Sharing/x.mp4", "local", "", "gdrive://Sharing/x.mp4"),
            # 2. an anchored local path (leading slash, or drive prefix) is always local,
never joined ?
            #    OS-independently (not via os.path.isabs, which differs on Windows+py3.13)
            ("/data/x.mp4", "gcs", "gs://r", "/data/x.mp4"),
            ("\\\\server\\share\\x.mp4", "gcs", "gs://r", "\\\\server\\share\\x.mp4"),
            ("C:\\v\\x.mp4", "gcs", "gs://r", "C:\\v\\x.mp4"),
            ("C:/v/x.mp4", "gcs", "gs://r", "C:/v/x.mp4"),
            # 3. a bare/relative name joins under a non-empty input_root
            ("x.mp4", "local", "gs://bucket/videos", "gs://bucket/videos/x.mp4"),
            (
                "sub/x.mp4",
                "local",
                "gs://bucket/videos/",
                "gs://bucket/videos/sub/x.mp4",
            ),  # root trailing slash
            # 4. input_backend prefixes a bare bucket/key when no input_root
            ("bucket/x.mp4", "gcs", "", "gcs://bucket/x.mp4"),
            ("bucket/x.mp4", "s3", "", "s3://bucket/x.mp4"),
            # 5. bare/relative local path ? today's behaviour, unchanged
            ("x.mp4", "local", "", "x.mp4"),
            ("rel/sub/x.mp4", "local", "", "rel/sub/x.mp4"),
            (
                "C:\\v\\x.mp4",
                "local",
                "",
                "C:\\v\\x.mp4",
            ),  # Windows path, default config: unchanged (OS-independent)
        ],
    )
    def test_resolve_input_uri(raw, ib, ir, expected):
        assert resolve_input_uri(raw, input_backend=ib, input_root=ir) == expected


    def test_resolve_input_uri_default_is_identity():
        for p in ("/data/clip.mp4", "rel/clip.mp4", "clip.mp4", "gs://b/k", "local://x"):
            assert resolve_input_uri(p) == p


    # ---------------------------------------------------------------- input_is_local
    def test_input_is_local_true_for_local_paths():
        assert input_is_local("/data/x.mp4") is True
        assert input_is_local("rel/x.mp4") is True
        assert input_is_local("local://x.mp4") is True


    def test_input_is_local_false_for_buckets():
        assert input_is_local("gs://b/x.mp4") is False
        assert input_is_local("s3://b/x.mp4") is False
        assert input_is_local("gdrive://Sharing/x.mp4") is False


    def test_input_is_local_follows_input_root():
        cfg = {"input_backend": "local", "input_root": "gs://bucket/videos"}
        assert input_is_local("x.mp4", cfg) is False  # bare name resolves into the bucket
        assert input_is_local("/abs/x.mp4", cfg) is True  # absolute path stays local
        assert input_is_local("gs://other/x.mp4", cfg) is False


    def test_input_is_local_accepts_pydantic_storage_config():
        from backend.config.models import StorageConfig
```

```
91
92          sc = StorageConfig(input_root="gs://bucket/videos")
93          assert input_is_local("x.mp4", sc) is False
94          assert input_is_local("/abs/x.mp4", sc) is True
95
96
97      def test_input_is_local_unknown_scheme_raises():
98          # An unsupported scheme is a ValueError (callers at the API/CLI boundary map it to a
clean error),
99          # never a silent True/False that could mislead a security gate.
100         for bad in ("https://h/x.mp4", "ftp://h/x.mp4", "file:///etc/passwd"):
101             with pytest.raises(ValueError):
102                 input_is_local(bad)
103
104
105     # ------------------------------------------------------------------ resolve_input_ref
106     def test_resolve_input_ref_local_and_remote():
107         local = resolve_input_ref("/data/x.mp4")
108         assert (local.backend, local.key) == ("local", "/data/x.mp4")
109         # local:// strips to the bare key (so callers stat ref.key, not the raw URI)
110         assert resolve_input_ref("local:///data/x.mp4").key == "/data/x.mp4"
111         remote = resolve_input_ref("x.mp4", {"input_root": "gs://bucket/videos"})
112         assert (remote.backend, remote.bucket, remote.key) == (
113             "gcs",
114             "bucket",
115             "videos/x.mp4",
116         )
117
118
119     def test_resolve_input_ref_unknown_scheme_raises():
120         with pytest.raises(ValueError):
121             resolve_input_ref("https://host/x.mp4")
122
123
124     # ------------------------------------------------------------------ acquire_local_input
125     def test_acquire_local_input_local_identity(tmp_path):
126         p = tmp_path / "v.mp4"
127         p.write_bytes(b"abc")
128         # Identity passthrough ? returns the SAME string, no copy, no temp dir.
129         assert acquire_local_input(str(p)) == str(p)
130
131
132     def test_acquire_local_input_relative_local_identity():
133         assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
134
135
136     def _capturing_fetch(captured):
137         def _f(uri, dst=None, *, config=None, overwrite=False):
138             captured["uri"] = uri
139             captured["dst"] = dst
140             captured["config"] = config
141             if dst:
142                 with open(dst, "wb") as fh:
143                     fh.write(b"remote-bytes")
144             return dst or uri
145
146         return _f
147
148
149     def test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
150         captured = {}
151         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
152         out = acquire_local_input(
153             "gs://bucket/videos/x.mp4", {"backend": "local", "gcs": {}}
154         )
```

```
155         assert captured["uri"] == "gs://bucket/videos/x.mp4"  # exact URI routed to fetch
156         assert os.path.basename(out) == "x.mp4"  # named from the ref
157         assert os.path.isfile(out) and open(out, "rb").read() == b"remote-bytes"
158         # The whole storage settings dict is forwarded so per-backend (gcs/s3) config
reaches fetch.
159         assert captured["config"] == {"backend": "local", "gcs": {}}
160
161
162     def test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
163         captured = {}
164         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
165         acquire_local_input("x.mp4", {"input_root": "gs://bucket/videos"})
166         assert captured["uri"] == "gs://bucket/videos/x.mp4"  # resolved BEFORE fetch
167
168
169     def test_acquire_local_input_never_fetches_for_local(monkeypatch):
170         def _boom(*a, **k):
171             raise AssertionError("fetch must not be called for a local input")
172
173         monkeypatch.setattr(storage, "fetch", _boom)
174         assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
175         assert (
176             acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
177             == "clip.mp4"
178         )
179
```

30. user (2026-07-02T04:50:56.953Z)

```
360         )  # explicit "run on my machine" ? no dispatch despite a cloud server
361
362
363     def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
364         """Server default is LOCAL (compute_target=''), but a per-request compute='cloud'
forces a Cloud
365         Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT +
a bucket)."""
366         db = Database(str(tmp_path / "t.db"))  # noqa: F841
367         monkeypatch.setattr(m, "db_instance", db)
368         bucket = tmp_path / "bucket"
369         bucket.mkdir()
370         cfg = AppConfig.model_validate(
371             {  # noqa: F841
372                 "deployment": {"compute_target": ""},  # server default = in-process
373                 "storage": {"backend": "local", "output_root": str(bucket)},
374             }
375         )
376         monkeypatch.setattr(m, "global_config", cfg)
377         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
378         fake_local = tmp_path / "fetched.mp4"
379         fake_local.write_bytes(b"FAKEVIDEOBYTES")
380         monkeypatch.setattr(
381             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
382         )
383         monkeypatch.setattr(
384             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
385         )
386         monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
387         monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
388         fake = _FakeCloudClient()
389         _inject_fake(monkeypatch, fake)
390
391         out = m._execute_processing(
392             m.global_config,
393             m.db_instance,
```

```
394             m.ProcessRequest(
395                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
396             ),
397         )
398         assert out["status"] == "success"
399         assert len(out["play_segments"]) == 2
400         assert (
401             len(fake.calls) == 1
402         )  # per-request override dispatched despite the LOCAL server default
403 
404 
405     def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
406         """compute='cloud' on a server NOT wired for it fails clearly (no silent local
fallback, no
407         dispatch) so the picker can surface 'cloud isn't available here'."""
408         db = Database(str(tmp_path / "t.db"))  # noqa: F841
409         monkeypatch.setattr(m, "db_instance", db)
```

31. user (2026-07-02T04:50:57.684Z)

```
535         )  # in-process rows, not cloud
536 
537 
538     def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
539         """REGRESSION (mirror): on a LOCAL-default server wired for cloud (env + bucket),
POST /api/process
540         with compute='cloud' must survive the boundary and dispatch to Cloud Run."""
541         db = Database(str(tmp_path / "t.db"))  # noqa: F841
542         monkeypatch.setattr(m, "db_instance", db)
543         bucket = tmp_path / "bucket"
544         bucket.mkdir()
545         cfg = AppConfig.model_validate(
546             {  # noqa: F841
547                 "deployment": {"compute_target": ""},  # server default = in-process
548                 "storage": {"backend": "local", "output_root": str(bucket)},
549             }
550         )
551         monkeypatch.setattr(m, "global_config", cfg)
552         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
553         fake_local = tmp_path / "fetched.mp4"
554         fake_local.write_bytes(b"FAKEVIDEOBYTES")
555         monkeypatch.setattr(
556             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
557         )
558         monkeypatch.setattr(
559             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
560         )
561         monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
562         monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
563         fake = _FakeCloudClient(video_id="vidBoundary")
564         _inject_fake(monkeypatch, fake)
565         app = m.create_app(m.global_config, db)
566         client = TestClient(app)
567 
568         r = client.post(
569             "/api/process",
570             json={
571                 "video_path": "gs://b/clip.mp4",
572                 "segmenter": "wasb_hybrid",
573                 "compute": "cloud",
574             },
575         )
576         assert r.status_code == 202
577         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
578         assert job["status"] == "done"
```

```
579         assert (
580             len(fake.calls) == 1
581         )  # 'cloud' honoured across the boundary ? dispatched despite local default
582         segs = db.query_play_segments(job["video_id"], {})
583         assert len(segs) == 2
584         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
585 
586 
587     # --- compute-path PROVENANCE (the validation method: prove which backend ran)
------------------
588 
589 
590     def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
591         """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run
execution name
592         (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the
intended path is
593         provable from the job result alone (no guessing from segment metadata)."""
594         _db, _bucket, monkeypatch = cloud_env
```

32. user (2026-07-02T04:51:05.455Z)

```
tests\test_async_jobs.py:391:def test_worker_dispatches_compile_kind_to_execute_compile(env):
tests\test_async_jobs.py:392:    """The claimed-job dispatcher routes kind='compile' to
_execute_compile (NOT _execute_processing),
tests\test_async_jobs.py:405:    mp.setattr(m, "_execute_compile", _fake_compile)
tests\test_api_assets.py:1:"""TestClient coverage for POST /api/assets ? "store this video in my
medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14).
tests\test_api_assets.py:46:        "/api/assets", json={"uploaded_path": str(up), "medium_uri":
"local://"}
tests\test_api_assets.py:72:        "/api/assets", json={"uploaded_path": str(up), "medium_uri":
scheme}
tests\test_api_assets.py:96:        "/api/assets", json={"uploaded_path": str(up), "medium_uri":
"gdrive://videos"}
tests\test_api_assets.py:114:        "/api/assets",
tests\test_api_assets.py:131:        "/api/assets", json={"uploaded_path": str(up),
"medium_uri": "s3://b/k"}
tests\test_api_assets.py:141:        "/api/assets",
tests\test_api_assets.py:144:    neither = client.post("/api/assets", json={"medium_uri":
"s3://b/k"})
tests\test_api_assets.py:151:        "/api/assets",
tests\test_api_assets.py:160:    r = client.post("/api/assets", json={"uploaded_path": str(up),
"medium_uri": ""})
tests\test_api_assets.py:180:        "/api/assets", json={"uploaded_path": str(up),
"medium_uri": "gdrive://videos"}
tests\test_api_assets.py:194:        "/api/assets", json={"uploaded_path": str(up),
"medium_uri": "s3://b/k"}
tests\test_api_assets.py:204:        "/api/assets", json={"uploaded_path": str(up),
"medium_uri": "ftp://nope/x"}
tests\test_compute_backend.py:202:def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
tests\test_compute_backend.py:203:    """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a
None dst raises) ? _read_json must
tests\test_compute_backend.py:222:    data = backend._read_json("gs://b/x.json")
tests\test_golden_real_fixtures.py:157:    prov = gf._read_json(fixtures / slug /
"provenance.json")
tests\test_golden_real_fixtures.py:291:    prov = gf._read_json(fixtures / expected["slug"] /
"provenance.json")
tests\test_golden_real_fixtures.py:319:    prov = gf._read_json(fixtures / ok["slug"] /
"provenance.json")
tests\test_golden_real_fixtures.py:541:    committed = gf._read_json(fdir / "expected.json")
tests\test_golden_real_fixtures.py:746:    prov = gf._read_json(fixtures / slug /
"provenance.json")
tests\test_golden_real_fixtures.py:794:            gf._read_json(fixtures / "fx_trim01" /
"expected.json"),
```

33. user (2026-07-02T04:51:12.900Z)

```
190         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
191         backend.run_infer(
192             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
193         )
194         backend.run_infer(
195             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
196         )
197         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
198         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
199         assert done1 != done2
200
201
202     def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
203         """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
204         pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
205         identity passthrough). Mimic a cloud backend that rejects dst=None."""
206         from backend.compute import cloud_run as cr
207
208         captured = {}
209
210         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
211             captured["dst"] = dst
212             if dst is None:
213                 raise ValueError(
214                     "cloud backend requires a dst path"
215                 )  # mirror GCSStorage/S3Storage
216             with open(dst, "w", encoding="utf-8") as f:
217                 f.write('{"video_id": "v", "returncode": 0}')
218             return dst
219
220         monkeypatch.setattr(cr, "fetch", fake_fetch)
221         backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
222         data = backend._read_json("gs://b/x.json")
223         assert (
224             captured["dst"] is not None
225         )  # the fix: a real dst is always passed (no silent {})
226         assert data == {"video_id": "v", "returncode": 0}
227
228
229     def test_cloud_run_polls_until_marker_appears(monkeypatch):
230         """The bucket-poll WAIT path (D7 scale-to-zero cold start): the marker is ABSENT for
the first
231         polls then appears ? poll_until must LOOP, not resolve on poll 0. The synchronous
fake elsewhere
232         never exercises this; here exists() is False twice then True."""
233         from backend.compute import cloud_run as cr
234
235         exists_calls = {"n": 0}
236
237         class _FakeStore:
238             def exists(self, ref):
239                 exists_calls["n"] += 1
```

34. user (2026-07-02T04:51:13.996Z)

```
100         assert body["medium"] == "gdrive" and body["copied"] is True
101         assert body["stored_uri"] == f"gdrive://videos/{vid}.mp4"
102         # the real copy landed on the mount filesystem
103         assert (mount / "videos" / f"{vid}.mp4").read_bytes() == up.read_bytes()
104
105
```

```
    def test_source_uri_already_at_rest_registers_without_copy(tmp_path, monkeypatch):
        client, db, _ = _client(tmp_path, monkeypatch)
        at_rest = _upload(tmp_path, name="already.mp4")  # a local path already at rest
        puts = []
        monkeypatch.setattr(
            m.storage, "put", lambda *a, **k: puts.append(a) or "should-not-be-called"
        )
        r = client.post(
            "/api/assets",
            json={"source_uri": str(at_rest), "medium_uri": "local://"},
        )
        assert r.status_code == 201
        body = r.json()
        assert body["copied"] is False and puts == []  # no copy (D14b)
        assert body["stored_uri"] == str(at_rest)
        assert body["video_id"] == compute_video_id(str(at_rest))


    # ---- guards ------------------------------------------------------------------

    def test_self_host_posture_rejects_when_jailed(tmp_path, monkeypatch):
        client, db, _ = _client(tmp_path, monkeypatch, allowed_video_root=str(tmp_path))
        up = _upload(tmp_path)
        r = client.post(
            "/api/assets", json={"uploaded_path": str(up), "medium_uri": "s3://b/k"}
        )
        assert r.status_code == 403
        assert "hosted" in r.json()["detail"].lower()


    def test_requires_exactly_one_source(tmp_path, monkeypatch):
        client, db, _ = _client(tmp_path, monkeypatch)
        up = _upload(tmp_path)
        both = client.post(
            "/api/assets",
            json={"uploaded_path": str(up), "source_uri": "s3://b/k", "medium_uri":
"s3://b/k"},
        )
        neither = client.post("/api/assets", json={"medium_uri": "s3://b/k"})
        assert both.status_code == 400 and neither.status_code == 400


    def test_missing_uploaded_file_404(tmp_path, monkeypatch):
        client, db, _ = _client(tmp_path, monkeypatch)
        r = client.post(
            "/api/assets",
            json={"uploaded_path": str(tmp_path / "nope.mp4"), "medium_uri": "s3://b/k"},
        )
        assert r.status_code == 404


    def test_upload_requires_medium_uri(tmp_path, monkeypatch):
        client, db, _ = _client(tmp_path, monkeypatch)
        up = _upload(tmp_path)
        r = client.post("/api/assets", json={"uploaded_path": str(up), "medium_uri": ""})
        assert r.status_code == 400
        assert "medium_uri" in r.json()["detail"]


    def test_free_space_507_on_low_disk_drive_medium(tmp_path, monkeypatch):
        mount = tmp_path / "drive"
        mount.mkdir()
        db = Database(str(tmp_path / "t.db"))
        cfg = AppConfig()
```

35. user (2026-07-02T04:51:51.412Z)

```
tests\test_api_endpoints.py:234:    mp.setattr(m.segmenter_registry, "get", lambda name:
_FakeSeg)
tests\test_api_endpoints.py:250:    mp.setattr(m.segmenter_registry, "get", lambda name:
_FakeSeg)
tests\test_api_endpoints.py:312:    mp.setattr(m.segmenter_registry, "get", lambda name:
_FakeSeg)
tests\test_api_endpoints.py:660:    mp.setattr(m.segmenter_registry, "get", lambda name:
_FakeSeg)
tests\test_api_endpoints.py:711:    mp.setattr(m.segmenter_registry, "get", lambda name:
_CaptureSeg)
tests\test_api_endpoints.py:738:    mp.setattr(m.segmenter_registry, "get", lambda name:
_CaptureSeg)
tests\test_api_endpoints.py:792:    mp.setattr(m.segmenter_registry, "get", lambda name:
_CaptureSeg)
tests\test_api_endpoints.py:823:        m.segmenter_registry, "get", lambda name: _FakeSeg
tests\test_api_endpoints.py:866:    mp.setattr(m.segmenter_registry, "get", lambda name:
_CaptureSeg)
tests\test_api_endpoints.py:893:    mp.setattr(m.segmenter_registry, "get", lambda name:
_FakeSeg)
tests\test_api_endpoints.py:917:    mp.setattr(m.segmenter_registry, "get", lambda name:
_FakeSeg)
tests\test_api_endpoints.py:937:    monkeypatch.setattr(m.segmenter_registry, "get", lambda
name: _CaptureSeg)
tests\test_api_endpoints.py:989:    monkeypatch.setattr(m.segmenter_registry, "get", lambda
name: _NoSeg)
tests\test_api_cloud_dispatch.py:263:    monkeypatch.setattr(m.segmenter_registry, "get", lambda
name: _InProcSeg)
tests\test_api_cloud_dispatch.py:346:    monkeypatch.setattr(m.segmenter_registry, "get", lambda
name: _InProcSeg)
tests\test_api_cloud_dispatch.py:513:    monkeypatch.setattr(m.segmenter_registry, "get", lambda
name: _InProcSeg)
tests\test_api_cloud_dispatch.py:617:    monkeypatch.setattr(m.segmenter_registry, "get", lambda
name: _InProcSeg)
tests\test_api_cloud_dispatch.py:639:    monkeypatch.setattr(m.segmenter_registry, "get", lambda
name: _InProcSeg)
tests\test_async_jobs.py:168:    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
tests\test_async_jobs.py:223:    mp.setattr(m.segmenter_registry, "get", lambda name: _EmptySeg)
tests\test_async_jobs.py:241:    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
tests\test_async_jobs.py:267:    mp.setattr(m.segmenter_registry, "get", lambda name: None)
tests\test_async_jobs.py:268:    mp.setattr(m.segmenter_registry, "list_available", lambda:
{"motion": "..."})
tests\test_async_jobs.py:285:    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
tests\test_async_jobs.py:332:    mp.setattr(m.segmenter_registry, "get", lambda name:
_RaisingSeg)
tests\test_async_jobs.py:350:    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
tests\test_async_jobs.py:458:    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
tests\test_base.py:7:class DummySegmenter(BasePlaySegmenter):
tests\test_base.py:65:    class ComplexCtorSegmenter(BasePlaySegmenter):
tests\test_base.py:105:    class BadDescSegmenter(BasePlaySegmenter):
tests\test_base.py:191:    class RaisingSegmenter(BasePlaySegmenter):
tests\test_generate_highlights.py:84:    ), patch.object(gh, "segmenter_registry",
seg_registry):
tests\test_generate_highlights.py:126:        gh, "segmenter_registry", seg_registry
tests\test_generate_highlights.py:163:    ), patch.object(gh, "segmenter_registry",
seg_registry):
tests\test_generate_highlights.py:260:        gh, "segmenter_registry", seg_registry
tests\test_generate_highlights.py:304:        gh, "segmenter_registry", seg_registry
tests\test_job_worker_loop.py:85:    mp.setattr(m.segmenter_registry, "get", lambda name:
_FakeSeg)
tests\test_job_worker_loop.py:163:    mp.setattr(m.segmenter_registry, "get", lambda name:
_WaitOnceSeg)
tests\test_job_worker_loop.py:274:    mp.setattr(m.segmenter_registry, "get", lambda name:
_BoomSeg)
```

```
tests\test_light_tier_no_torch.py:37:from backend.pipeline.segmenters.base import
segmenter_registry

[Showing results with pagination = limit: 40]
```

36. user (2026-07-02T04:52:04.559Z)

```
100         assert {
101             "id",
102             "video_path",
103             "video_id",
104             "status",
105             "progress",
106             "error",
107             "result",
108         } <= cols
109         assert {"policy", "next_poll_at"} <= cols  # v3 self-scheduling columns
110         assert {
111             "locale",
112             "compute",
113             "kind",
114             "params",
115         } <= cols  # v7 locale, v8 compute, v9 compile job
116
117
118     def test_migration_upgrades_v1_db_in_place(tmp_path):
119         path = str(tmp_path / "v1.db")
120         conn = sqlite3.connect(path)
121         conn.execute(
122             "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
123             "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)"
124         )
125         # A genuine v1 DB also has candidate_segments (created in the version<1 block);
include it
126         # so the v4 personalization ALTER (which adds user_id to that table) has a table to
alter.
127         conn.execute(
128             "CREATE TABLE candidate_segments (id INTEGER PRIMARY KEY AUTOINCREMENT, "
129             "video_id TEXT NOT NULL, detector TEXT NOT NULL, start_time REAL NOT NULL, "
130             "end_time REAL NOT NULL, duration REAL NOT NULL, stats TEXT)"
131         )
132         conn.execute("PRAGMA user_version = 1")
133         conn.commit()
134         conn.close()
135
136         db = Database(path)  # ladder should take it 1 ? 9
137         with db._get_connection() as c:
138             assert c.execute("PRAGMA user_version").fetchone()[0] == 9
139             assert c.execute(
140                 "SELECT name FROM sqlite_master WHERE type='table' AND name='jobs'"
141             ).fetchone()
142             # Restore v5 personalization coverage (user_models table + user_id on
candidate_segments).
143             # These were dropped in prior edit; add (do not replace) per review.
144             tables = {
145                 r[0]
146                 for r in c.execute(
147                     "SELECT name FROM sqlite_master WHERE type='table'"
148                 ).fetchall()
149             }
150             assert "user_models" in tables
151             cand_cols = {
152                 r["name"]
153                 for r in c.execute("PRAGMA table_info(candidate_segments)").fetchall()
154             }
```

```
            assert "user_id" in cand_cols


    def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
        """Item 5 positive: direct CLI-style path (m._execute_processing) vs job-worker path
        (submit + _InlineExecutor leading to _run_claimed_job / drain, plus explicit
_run_claimed_job)
        produce equivalent outcomes for the same ProcessRequest shape. Exercises success,
        Failure, and empty-run branches. Real DB queries using vid (no self-compares or
tautologies).
        """
        client, db, tpath, mp = env
        vid_file = _video(tpath)
        vid = m.compute_video_id(str(vid_file))
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        req = m.ProcessRequest(video_path=str(vid_file), segmenter="x")

        # Direct CLI-style path
        direct_out = m._execute_processing(m.global_config, m.db_instance, req)
        segs = db.query_play_segments(vid, {})
        assert len(segs) >= 1, f"direct path must affect segments for vid {vid}"
        assert direct_out.get("result", {}).get("status") in (
            None,
            "success",
            "processed",
        ) or "play_segments" in str(direct_out)

        # Job-worker path via public API submit (exercises job creation + inline executor +
internal _run_claimed_job)
        rj = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
        )
        job_out = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
        segs_j = db.query_play_segments(vid, {})
        assert job_out["result"]["status"] == "success"
        assert len(segs_j) >= 1

        # Explicit job-worker via _run_claimed_job (create + mark running + drive the
claimed runner)
        vid_file2 = _video(tpath, name="m2.mp4")
        vid2 = m.compute_video_id(str(vid_file2))
        jid = f"parity-{vid2[:8]}"
        db.create_job(jid, str(vid_file2))
        db.mark_job_running(jid)
        job_row = db.get_job(jid)
        m._run_claimed_job(job_row, m.global_config, m.db_instance)
        segs2 = db.query_play_segments(vid2, {})
        assert len(segs2) >= 1, (
            f"_run_claimed_job path must produce segments for vid {vid2}"
        )

        # Failure branch (direct + job submit)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
        req_f = m.ProcessRequest(video_path=str(vid_file))
        direct_f = m._execute_processing(m.global_config, m.db_instance, req_f)
        rf = client.post("/api/process", json={"video_path": str(vid_file)})
        jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
        assert (
            direct_f.get("result", {}).get("status") == "failed"
            or jobf["result"]["status"] == "failed"
        )

        # Empty-run branch (success verdict but zero play_segments from this execution)
```

```
        class _EmptySeg:
            def __init__(self, db, cfg):
                self.db = db

            def process_video(self, video_id, video_path, *a, **k):
                return []

        mp.setattr(m.segmenter_registry, "get", lambda name: _EmptySeg)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        req_e = m.ProcessRequest(video_path=str(vid_file))
        de = m._execute_processing(m.global_config, m.db_instance, req_e)
        re = client.post("/api/process", json={"video_path": str(vid_file)})
        je = client.get(f"/api/jobs/{re.json()['job_id']}").json()
        assert de.get("result", {}).get("status") in (None, "success")
        assert je["result"]["status"] == "success"
        # (reingest safety for prior on empty/fail covered in dedicated tests)


    # --- submit-time fast-fail ------------------------------------------------------------


    def test_submit_returns_202_with_job_id(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
        r = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
        )
        assert r.status_code == 202
        body = r.json()
        assert body["job_id"]
        assert body["status"] == "queued"
        assert body["poll"].endswith(body["job_id"])
```

37. user (2026-07-02T04:52:10.866Z)

```
      """Async job model (DEFERRED_HARDENING_PLAN #16).

      Contract under test:
      - POST /api/process ? 202 + job_id (fast-fail 4xx still happens at submit time)
      - GET /api/jobs/{id} ? {status, progress, result, reports, ...}
      - job `status` = EXECUTION state (queued|running|done|failed); the pipeline's own
        verdict (e.g. failed validation) lives in result["status"].
      - worker crash restores pre-run segments (destroy-AFTER-confirm) and marks the job
failed
      - startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
      - migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in place
      """

      import json
      import sqlite3
      import time
      from concurrent.futures import ThreadPoolExecutor

      import pytest

      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
      from fastapi.testclient import TestClient

      import backend.main as m
      from backend.config.models import AppConfig
      from backend.infrastructure.database import Database
      from backend.pipeline.validators.base import ValidationResult

```

```
    class _InlineExecutor:
        """submit() runs the fn synchronously ? deterministic tests, no threads."""

        def submit(self, fn, *args, **kwargs):
            fn(*args, **kwargs)


    @pytest.fixture
    def env(tmp_path, monkeypatch):
        db = Database(str(tmp_path / "t.db"))
        cfg = AppConfig()
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        app = m.create_app(cfg, db)
        client = TestClient(app)
        return client, db, tmp_path, monkeypatch


    def _video(tmp_path, name="m.mp4"):
        p = tmp_path / name
        p.write_bytes(b"not-a-real-video-but-hashable")
        return p


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    def _fail():
        return [
            ValidationResult(
                validator_name="blur", passed=False, message="blurry", details={}
            )
        ]


    class _FakeSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            sid = self.db.add_play_segment(video_id, 0.0, 5.0)
            return [
                {
                    "id": sid,
                    "start_time": 0.0,
                    "end_time": 5.0,
                    "duration": 5.0,
                    "metadata": {},
                }
            ]


    class _RaisingSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            self.db.add_play_segment(video_id, 9.0, 10.0)  # half-written row pre-crash
            raise RuntimeError("boom")


    # --- migration -------------------------------------------------------------


```

```
93      def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
94          db = Database(str(tmp_path / "fresh.db"))
95          with db._get_connection() as conn:
96              # v9 = async compile (kind/params); v8 = compute-picker; v7 = UI locale; v6 =
cost ledger;
97              # jobs at v2/v3, user_models v5.
98              assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
99              cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
100         assert {
```

38. user (2026-07-02T04:52:37.253Z)

```
700         if sc is None:
701             return {}
702         return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
703
704
705     def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
706         """The durable, MD5-keyed destination inside the chosen medium:
``<medium>/<video_id><ext>``."""
707         return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
708
709
710     def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
711         """The LOCAL directory a ``put`` into this medium would write into (so a free-space
BLOCK can be
712         enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
713         ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount."""
714         if medium_ref.backend == "local":
715             return medium_ref.key or "."
716         if medium_ref.backend == "gdrive":
717             from backend.storage.capabilities import _gdrive_mount_root
718
719             root = _gdrive_mount_root(storage_settings)
720             if not root:
721                 return None
722             rel = medium_ref.key.lstrip("/")
723             return os.path.join(root, *rel.split("/")) if rel else root
724         return None  # s3 / gcs ? a bucket has no readable local capacity
725
726
727     @app.post("/api/assets", status_code=201)
728     def create_asset(req: AssetRequest, request: Request):
729         """Store a video into the user's chosen medium and register it as a library asset
(P4, D14).
730
731         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
is set ? the
732         hosted alpha jails inputs and has no per-user medium credentials, so it stays
upload-to-local.
733         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
local file from
734         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
uploaded file +
735         a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
local upload
736         stays as the working copy; a ``local`` medium keeps the upload in place (it's
already on this box);
737         a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
whole-file
738         MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
in
739         ``GET /api/videos``."""
740         try:
```

```
741             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
742         except edge_auth.EdgeAuthError as e:
743             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
744 
745         cfg = request.app.state.config
746         sec = getattr(cfg, "security", None)
747         if getattr(sec, "allowed_video_root", None):
748             raise HTTPException(
749                 status_code=403,
750                 detail=(
751                     "storing into a custom medium is disabled on this hosted deployment "
752                     "(security.allowed_video_root is set); upload to local instead."
753                 ),
754             )
755         if bool(req.uploaded_path) == bool(req.source_uri):
756             raise HTTPException(
757                 status_code=400,
758                 detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
759             )
760         if req.uploaded_path and not req.medium_uri:
761             raise HTTPException(
762                 status_code=400, detail="'medium_uri' is required when uploading a file."
763             )
764 
765         storage_settings = _storage_settings(cfg)
766 
767         # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
D3).
768         if req.uploaded_path:
769             try:
770                 validate_input_path(req.uploaded_path, allowed_root=None)
771                 src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
772             except ValueError as e:
773                 raise HTTPException(status_code=400, detail=str(e)) from e
774             if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
```

39. user (2026-07-02T04:52:38.200Z)

```
1545:def _resolve_compile(
1546-    db, cfg, video_id: str, segment_ids: List[int], output_filename: str
1547-):
1548-    """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
1549-    row could change between submit and run). Verifies the video exists, the requested
segments match
1550-    and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known shot_count
are exempt
1551-    so manual/legacy picks can't silently vanish), and the output name is traversal-safe.
Raises
1552-    ``_CompileError(404/400)``. Returns ``(video_row, kept_segments, safe_out_name)``."""
1553-    video = db.get_video(video_id)
1554-    if not video:
1555-        raise _CompileError(404, "Indexed video record not found.")
1556-    # Reject path traversal / absolute names (os.path.join would otherwise write anywhere
on disk).
1557-    try:
1558-        out_name = safe_output_filename(output_filename)
1559-    except ValueError as e:
1560-        raise _CompileError(400, str(e)) from e
1561-
1562-    all_segments = db.query_play_segments(video_id, {})
1563-    matched = [s for s in all_segments if s["id"] in segment_ids]
1564-    if not matched:
1565-        raise _CompileError(400, "No matching play segments found to stitch.")
1566-
```

```
1567-    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1568-    kept = []
1569-    for s in matched:
1570-        meta = s.get("metadata") or {}
1571-        sc = meta.get("shot_count") if isinstance(meta, dict) else None
1572-        try:
1573-            if sc is not None and int(sc) < stitching.min_shot_count:
1574-                continue
1575-        except (TypeError, ValueError):
1576-            pass  # unparseable count -> treat as unknown, keep
1577-        kept.append(s)
1578-    if len(kept) < len(matched):
1579-        logger.info(
1580-            "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1581-            stitching.min_shot_count,
1582-            len(matched) - len(kept),
1583-            len(matched),
1584-        )
1585-    if not kept:
1586-        raise _CompileError(
1587-            400,
1588-            f"All {len(matched)} matching segments fall below "
1589-            f"stitching.min_shot_count={stitching.min_shot_count}.",
1590-        )
```

40. user (2026-07-02T04:52:45.964Z)

```
1       import argparse
2       import csv
3       import json
4       import logging
5       import math
6       import os
7       import shutil
8       import socket
9       import sys
10      import tempfile
11      import threading
12      import uuid
13      import webbrowser
14      from importlib.metadata import PackageNotFoundError
15      from importlib.metadata import version as _pkg_version
16      from urllib.parse import quote
17
18      import uvicorn
19      from dotenv import load_dotenv
20
21      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
22      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
23      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
24      # stdlib-only import ? keeps this pre-pydantic line light.
25      from backend.utils.app_home import env_file_to_load, resolve_output_dir
26
27      load_dotenv(env_file_to_load())
28
29      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
30      logging.basicConfig(
31          level=logging.INFO,
32          format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
33          datefmt="%H:%M:%S",
```

```
    )
    logger = logging.getLogger(__name__)
    from typing import Any, Dict, List, Optional, Tuple

    from fastapi import FastAPI, HTTPException, Request
    from fastapi.middleware.cors import CORSMiddleware
    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
    from fastapi.staticfiles import StaticFiles
    from pydantic import BaseModel
    from starlette.middleware.trustedhost import TrustedHostMiddleware

    from backend import (
        billing,  # M8: per-job cost ledger (USD internal / INR display)
        storage,  # M2: URI-aware input acquisition (local = identity passthrough)
    )
    from backend.api import (
        auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
    )
    from backend.config import (
        AppConfig,
        StartupCheckError,
        StitchingConfig,
        enforce_startup_checks,
        write_light_default_config,  # P2b: batteries-included first-run config seeding
    )
    from backend.config import (  # new typed config
        load_config as load_app_config,
    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import registry
    from backend.providers import provider_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure  # standardized error/result contract
    from backend.storage.capabilities import (
        storage_capabilities,  # P3: secret-free "which media can this box use" report
    )
    from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
        ffmpeg_bin,
        ffprobe_bin,
    )
    from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import (
        output_base,
        resolve_under_root,
        safe_output_filename,
        validate_input_path,
    )
    from backend.utils.redact import (
        redact_secrets,  # P0b: mask secret-shaped fields in /api/config
    )
    from backend.utils.single_instance import acquire_lock  # P0e: one instance per database

    app = FastAPI(title="Sports Highlight Indexer API")


    def create_app(config: AppConfig, db: Database) -> FastAPI:
        """Factory: inject shared state via ``app.state`` so route handlers access it
through
        ``request.app.state`` ? eliminating the Optional module-level globals that forced
        ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.

```

```
98          Also sets the module-level ``global_config`` / ``db_instance`` for transitional
99          compatibility (helpers and tests still reference them). Route handlers use
100         ``request.app.state`` exclusively."""
101         global global_config, db_instance
102         global_config = config
103         db_instance = db
104         app.state.config = config
105         app.state.db = db
106         return app
107
108
109     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
110         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
111         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
112         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
113         src = os.environ if env is None else env
114         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
115         origins = [o.strip() for o in raw.split(",") if o.strip()]
116         return origins or ["*"]
117
118
119     # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
120     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
```

41. user (2026-07-02T04:52:59.047Z)

```
1       [mypy]
2       # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3       # modules and pins an explicit quarantine list below ? not a typing crusade.
4       # Ratchet: when you touch a quarantined module, try removing its entry.
5       ignore_missing_imports = True
6       # backend/ is a namespace package (no root __init__.py) ? required so mypy
7       # maps files to backend.* module names instead of duplicating top-level names.
8       explicit_package_bases = True
9
10      # --- Quarantine (known-loose modules, each with a reason) ---
11
12      # Optional module globals (db_instance/global_config set by create_app() / main()).
13      # Route handlers use request.app.state exclusively; helpers take explicit params.
14      # backend.api.job_worker lifted from quarantine (now takes explicit config/db params).
15      [mypy-backend.main]
16      ignore_errors = True
17
18      # Worker dispatcher: composes the SAME validator/segmenter/report building blocks as
19      # backend.main, which are typed `config: dict` but receive the AppConfig migration shim
20      # at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
21      # until the typed-config migration removes the dict?AppConfig duality everywhere.
22      [mypy-backend.worker.__main__]
23      ignore_errors = True
24
25      # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
26      # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
27      # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
28      # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
29      # annotation) ? ratchet those out in a focused typing pass, not here.
30      # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
31      [mypy-backend.pipeline.segmenters.yolo_hybrid]
32      ignore_errors = True
33
34      [mypy-backend.pipeline.segmenters.gemini]
```

```
35      ignore_errors = True
36
37      # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
38      [mypy-backend.pipeline.segmenters.base]
39      ignore_errors = True
40
```

42. user (2026-07-02T04:52:59.471Z)

```
1       # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2       # bury blame history for zero behavior value).
3       #
4       # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5       # warnings (W) + bugbear (B) + isort (I ? consistent import ordering).
6       target-version = "py38"
7
8       # scratch/ = throwaway debugging scripts that may hit real APIs; it is also
9       # excluded from pytest collection (see pytest.ini).
10      extend-exclude = ["scratch"]
11
12      [lint]
13      extend-select = ["W", "B", "I"]
14
15      [lint.per-file-ignores]
16      # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
17      # sys.path) before importing backend modules ? import order is load-bearing.
18      "backend/main.py" = ["E402"]
19      "scripts/run_phase3_eval.py" = ["E402"]
20      "scripts/run_process_and_stitch.py" = ["E402"]
21
```

43. assistant (2026-07-02T04:53:06.251Z)

I have the full picture. Now setting up the isolated worktree per the branching safety protocol.

44. user (2026-07-02T04:53:12.403Z)

```
Preparing worktree (new branch 'fix/reclaim-input-scratch')
branch 'fix/reclaim-input-scratch' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe
details) (#413)
fix/reclaim-input-scratch
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

45. assistant (2026-07-02T04:53:22.959Z)

Let me check pytest.ini for exclusion patterns, then start implementing.

46. user (2026-07-02T04:53:24.466Z)

```
1       [pytest]
2       # When `pytest` is invoked without an explicit path, only collect from tests/.
3       # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4       # and should never run as part of the regular test suite.
5       testpaths = tests
6
```

47. user (2026-07-02T04:53:37.551Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

48. user (2026-07-02T04:53:45.151Z)

ok

49. user (2026-07-02T04:53:53.603Z)

```
"""Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.

URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
Google Drive (Google Drive for Desktop) as plain files.

Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
passthrough. See ``docs/DECOUPLED_COMPUTE/``.
"""

from __future__ import annotations

import os
import tempfile
from typing import Any, Dict, Iterator, Optional

from backend.storage.base import StorageBackend
from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

__all__ = [
    "StorageBackend",
    "StorageRef",
    "StorageStat",
    "parse_uri",
    "resolve_input_uri",
    "get_storage",
    "fetch",
    "put",
    "list_uris",
    "acquire_local_input",
    "input_is_local",
    "resolve_input_ref",
]


def _storage_dict(storage_config: Any) -> Dict[str, Any]:
    """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
    if storage_config is None:
        return {}
    if hasattr(storage_config, "model_dump"):
        return storage_config.model_dump()  # type: ignore[no-any-return]
    return dict(storage_config)


def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
    """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
    config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
    callers at the API/CLI boundary catch that and return a clean client error (never a
500).

    Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
    ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
```

```
55          sc = _storage_dict(storage_config)
56          uri = resolve_input_uri(
57              raw,
58              input_backend=sc.get("input_backend", "local"),
59              input_root=sc.get("input_root", ""),
60          )
61          return parse_uri(uri)
62
63
64      def input_is_local(raw: str, storage_config: Any = None) -> bool:
65          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
66          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
67          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
68          ``resolve_input_ref``)."""
69          return resolve_input_ref(raw, storage_config).backend == "local"
70
71
72      def acquire_local_input(
73          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
74      ) -> str:
75          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
76
77          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
78          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
79          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
80          path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
81          regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
82          worker's ``cmd_infer`` already ships."""
83          sc = _storage_dict(storage_config)
84          ref = resolve_input_ref(raw, sc)
85          if ref.backend == "local":
86              return (
87                  ref.key
88              )  # identity ? no temp, no copy, byte-identical to passing the path directly
89          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
90          return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
91
92
93      def _backend_class(name: str):
94          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
95          if name == "local":
```

50. user (2026-07-02T04:54:07.672Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-
reclaim/backend/storage/__init__.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

51. user (2026-07-02T04:54:27.876Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-
reclaim/backend/storage/__init__.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

52. assistant (2026-07-02T04:54:35.589Z)

Now the sweep module.

53. assistant (2026-07-02T04:54:58.545Z)

```
"""Age-based startup sweep for orphaned fetched-input scratch dirs (R1-17).

``acquire_local_input`` downloads every remote input (``gs://``/``s3://``/``gdrive://``) into a
fresh ``rally-input-*`` mkdtemp, and the worker's ``cmd_infer`` uses ``rally-worker-*`` the same
way. The per-job scope (``acquire_local_input_scoped``) reclaims those on the normal
success/failure paths, but a crash / SIGKILL / pre-scope leak leaves multi-GB video copies under
the system temp dir FOREVER on the shipping platform (Windows never sweeps ``%TEMP%`` ? the old
"relies on OS temp reclamation" premise was false). This sweep runs once in the server/CLI boot
path and deletes only:

- **directories** (never plain files),
- whose basename carries one of OUR ownership-marker prefixes (``SWEEP_PREFIXES``),
- directly under the system temp dir (or an explicit ``temp_root``),
- older than the age threshold (default 3 days ? far beyond any legitimate in-flight job, so a
  running fetch/segment can never be swept out from under itself).

DEFAULT-ON with a config kill-switch (``storage.scratch_sweep_enabled=false``) and an age knob
(``storage.scratch_sweep_max_age_days``). Every removal is logged. Never raises ? a janitor must
not be able to break boot.
"""

from __future__ import annotations

import logging
import os
import shutil
import tempfile
import time
from typing import Any, List, Optional

from backend.storage import SCRATCH_INPUT_PREFIX

logger = logging.getLogger(__name__)

#: Ownership markers: only dirs whose basename starts with one of these are ever touched.
#: ``rally-input-`` = acquire_local_input fetch scratch; ``rally-worker-`` = the worker
#: ``cmd_infer`` scratch (``backend/worker/__main__.py``).
SWEEP_PREFIXES = (SCRATCH_INPUT_PREFIX, "rally-worker-")

#: Default age threshold (days) before an orphaned scratch dir is reclaimed.
DEFAULT_MAX_AGE_DAYS = 3.0


def sweep_orphan_scratch(
    temp_root: Optional[str] = None,
    *,
    max_age_days: float = DEFAULT_MAX_AGE_DAYS,
    now: Optional[float] = None,
) -> List[str]:
    """Delete orphaned scratch dirs under ``temp_root`` (default: the system temp dir) older
    than ``max_age_days``; return the paths removed. Age = the dir's own mtime (set at mkdtemp
    and bumped when a fetch lands a file in it), so anything younger than the threshold ? in
    particular every in-flight job ? is left alone. Best-effort per entry and never raises:
    an unstattable/undeletable entry is skipped (and retried at the next boot)."""
    root = temp_root or tempfile.gettempdir()
    cutoff = (now if now is not None else time.time()) - max_age_days * 86400.0
    removed: List[str] = []
    try:
        entries = os.listdir(root)
```

```
    except OSError as e:
        logger.warning("[SCRATCH-SWEEP] cannot list %s (%s); skipping.", root, e)
        return removed
    for name in entries:
        if not name.startswith(SWEEP_PREFIXES):
            continue
        path = os.path.join(root, name)
        try:
            st = os.lstat(path)
        except OSError:
            continue  # vanished / unreadable ? not ours to force
        if not os.path.isdir(path) or os.path.islink(path):
            continue  # only real directories carry our mkdtemp marker
        if st.st_mtime >= cutoff:
            continue  # too young ? could belong to an in-flight job
        try:
            shutil.rmtree(path)
        except OSError as e:
            logger.warning("[SCRATCH-SWEEP] could not remove %s (%s); skipped.", path, e)
            continue
        removed.append(path)
        logger.info(
            "[SCRATCH-SWEEP] removed orphaned scratch %s (age %.1f days).",
            path,
            ((now if now is not None else time.time()) - st.st_mtime) / 86400.0,
        )
    return removed


def maybe_sweep_scratch(config: Any) -> List[str]:
    """Boot-path wrapper: run the sweep under the config knobs. DEFAULT-ON (the sweep only ever
    touches this tool's own prefix-marked dirs, aged well past any live job); kill-switch =
    ``storage.scratch_sweep_enabled=false``; threshold = ``storage.scratch_sweep_max_age_days``.
    Returns the removed paths. Never raises ? a failed sweep is logged and swallowed."""
    try:
        sc = getattr(config, "storage", None)
        if not getattr(sc, "scratch_sweep_enabled", True):
            return []
        max_age = float(
            getattr(sc, "scratch_sweep_max_age_days", DEFAULT_MAX_AGE_DAYS)
        )
        removed = sweep_orphan_scratch(max_age_days=max_age)
        if removed:
            logger.info(
                "[STARTUP] scratch sweep reclaimed %d orphaned dir(s) older than %s day(s).",
                len(removed),
                max_age,
            )
        return removed
    except Exception as e:  # noqa: BLE001 ? janitor must never break boot
        logger.warning("[STARTUP] scratch sweep skipped (non-fatal): %s", e)
        return []
```

54. user (2026-07-02T04:54:58.578Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-
reclaim/backend/storage/scratch_sweep.py (file state is current in your context ? no need to
Read it back)
```

55. assistant (2026-07-02T04:55:03.171Z)

Now the config knobs.

56. user (2026-07-02T04:55:04.888Z)

```
380         input_root: str = ""
381         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
382         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
383         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
384         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
385         # the legacy flat layout is used unchanged.
386         single_folder_per_video: bool = False
387         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
388         workspace_root: str = ""
389         # Cloud namespace prefix under the root so per-video folders sit tidily beside
390         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
391         workspace_prefix: str = "workspaces"
392         local: dict[str, Any] = Field(default_factory=dict)
393         s3: dict[str, Any] = Field(default_factory=dict)
394         gcs: dict[str, Any] = Field(default_factory=dict)
395         gdrive: dict[str, Any] = Field(default_factory=dict)
396         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
397         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
398         gdrive_api: dict[str, Any] = Field(default_factory=dict)
399
```

57. user (2026-07-02T04:55:16.585Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-
reclaim/backend/config/models.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

58. assistant (2026-07-02T04:55:21.845Z)

Now `cloud_run._read_json`.

59. user (2026-07-02T04:55:29.346Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

60. user (2026-07-02T04:55:38.252Z)

```
200                 execution=str(execution or ""),
201             )
202
203         # --- storage-backed completion signal (bucket-poll) ---
204         def _read_marker(self, done_uri: str) -> Optional[dict]:
205             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206             ref = parse_uri(done_uri)
207             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208                 return None
209             return (
210                 self._read_json(done_uri) or {}
211             )  # empty dict still terminates the poll (present == done)
212
213         def _read_json(self, uri: str) -> dict:
214             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
215             # raises ValueError ? unlike local's identity passthrough. Without an explicit
```

```
dst this read
216             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
217             try:
218                 ref = parse_uri(uri)
219                 dst = os.path.join(
220                     tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
221                 )
222                 local = fetch(uri, dst, config=self._storage_config)
223                 with open(local, "r", encoding="utf-8") as f:
224                     data: Any = json.load(f)
225                 return data if isinstance(data, dict) else {}
226             except Exception:
227                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
228                 return {}
229
```

61. user (2026-07-02T04:55:51.608Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-
reclaim/backend/compute/cloud_run.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

62. user (2026-07-02T04:56:03.512Z)

```
21:requires-python = ">=3.10"
```

63. user (2026-07-02T04:56:11.936Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-
reclaim/backend/compute/cloud_run.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

64. assistant (2026-07-02T04:56:54.134Z)

```
Let me check the ReportConfig default ? it matters for whether reports land in scratch.
```

65. user (2026-07-02T04:56:56.570Z)

```
272-    default_highlights_name: str = "highlights.mp4"
273-    db_name: str = "sports_indexer.db"
274-    # Directory where the DB, highlight reels, and processed clips are written.
275-    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
276:    output_dir: str = "output"
277-
278-
279-class WatermarkConfig(BaseModel):
280-    """Branding overlay burned into each highlight clip during the per-clip re-encode
--
365-    **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
366-    (boto3/google auth chains supply credentials). See ``backend/storage/``."""
367-
368-    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
369:    # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
370-    output_root: str = ""
371-    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
372-    # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is read
IN PLACE
373-    # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
```

```
bare/relative
--
391-    # it only ever deletes this tool's own prefix-marked dirs, aged well past any live job
392-    # (Windows never sweeps %TEMP% on its own). Kill-switch: set
scratch_sweep_enabled=false.
393-    scratch_sweep_enabled: bool = True
394-    scratch_sweep_max_age_days: float = 3.0
395:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
396-    workspace_root: str = ""
397-    # Cloud namespace prefix under the root so per-video folders sit tidily beside
398-    # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
399-    workspace_prefix: str = "workspaces"
```

66. user (2026-07-02T04:57:05.352Z)

```
384:    # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5). When
OFF
482:    # the alias never deprecates underneath us. Eval reports record the run date alongside.
```

67. user (2026-07-02T04:57:17.197Z)

```
1161        notify = progress or (lambda stage: None)
1162
1163        notify("hashing")
1164        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
1165        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
1166        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
1167        # consumers (hash / validators / segmenter) use the resolved local path.
1168        local_video = storage.acquire_local_input(
1169            req.video_path, getattr(config, "storage", None)
1170        )
1171        video_id = compute_video_id(local_video)
1172        was_reingest = db.get_video(video_id) is not None
1173
1174        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
1175        notify("validating")
1176        logger.info(f"Running validations for {req.video_path}...")
1177        val_results, val_context = registry.run_all_with_context(local_video, config)
1178        failures = [r for r in val_results if not r.passed]
1179
1180        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181        default_seg = getattr(
1182            getattr(config, "indexing", None), "default_segmenter", "motion"
1183        )
1184        seg_name = req.segmenter or default_seg
1185        segmenter_actual = None
1186        segmentation_ran = False
1187        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190        compute_actual: Optional[str] = None
1191        response: Optional[Dict[str, Any]] = None
1192        try:
1193            if failures:
1194                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1195                # status; it no longer destroys the video's prior good segments.
```

68. user (2026-07-02T04:57:35.601Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-reclaim/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

69. assistant (2026-07-02T04:57:42.260Z)

Now the end of `_execute_processing`'s finally block.

70. user (2026-07-02T04:57:47.301Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-reclaim/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

71. assistant (2026-07-02T04:57:55.841Z)

Site 2: `_execute_compile`.

72. user (2026-07-02T04:58:04.062Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-reclaim/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

73. assistant (2026-07-02T04:58:08.947Z)

Site 3: `create_asset`.

74. user (2026-07-02T04:58:13.356Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-reclaim/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

75. user (2026-07-02T04:58:27.448Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-scratch-reclaim/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

76. assistant (2026-07-02T04:58:28.177Z)

You've hit your session limit · resets 1:10pm (Asia/Calcutta)